Web aplikacije, Predavanje 01

Klijent, poslužitelj i HTTP

Fakultet informatike, Sveučilište Jurja Dobrile u Puli

izv. prof. dr. sc. Nikola Tanković

Ovo predavanje polazi od poznatog dohvata podataka pomoću Firebasea. Od poziva biblioteke postupno dolazimo do mrežne poruke i vlastitog poslužitelja; HTTP i TCP nisu predznanje. Predznanje su JavaScript, razvoj frontenda i korištenje Firebasea. Prvo razlikujemo opažanje od pretpostavke, zatim povezujemo komponente, poruke, stanje i kvarove. Primjeri su nastavni i ne određuju temu studentskog projekta.

1 Stranica se otvorila. Gdje su proizvodi?

Vrijeme za nastavu: 1–3 min (2 min).

Zamislimo trgovinu koja u preglednik učitava Vue aplikaciju. Zaglavlje, izbornik i gumb za osvježavanje vidljivi su, ali kartice proizvoda nedostaju. Početno opažanje govori samo da je preglednik uspio prikazati dio sučelja. Ne govori je li naknadni zahtjev za podatke uopće poslan, je li odgovor primljen ni je li ga komponenta pravilno obradila. Jednako su mogući problem JavaScripta, spora udaljena usluga, nedostupna baza i valjan prazan rezultat koji je loše prikazan.

Ovu situaciju koristimo kao istraživački problem. Najprije bilježimo hipoteze, a zatim određujemo opažanje koje bi ih razlikovalo. Studenti zasad opisuju što bi htjeli provjeriti običnim riječima. Kasnije ćemo u alatu preglednika vidjeti je li poruka poslana i što je stiglo natrag. Otvaranje stranice i dohvat podataka odvojeni su događaji.

Sučelje je vidljivo, ali umjesto proizvoda piše **„Učitavanje...”**.

Što smijemo zaključiti?

A Usluga za dohvat podataka sigurno ne radi.
B Baza je prazna.
C Trebamo još dokaza.

Za nastavnika: Glasajte A/B/C i navedite jedan dokaz koji nedostaje.

Očekivano obrazloženje: C. Prihvatite opis običnim riječima: je li dohvat pokrenut, jesu li podaci stigli i je li ih sučelje prikazalo? Stručne nazive tek uvodimo.

2 Ishodi i predznanje

Vrijeme za nastavu: 3–4 min (1 min).

Nastavljamo na frontend znanje, ali promatramo aplikaciju iz perspektive zahtjeva. Ne učimo ponovno sintaksu JavaScripta niti konfiguriramo puni framework. Poznat je prikaz podataka u sučelju. Pojmove klijent, poslužitelj, zahtjev, odgovor, ruta i mrežna veza uvodimo postupno, bez pretpostavke da su studenti već pratili komunikaciju. Svaki odgovor treba povezati s komponentom koja nešto zna ili radi.

Do kraja sata student treba moći nacrtati put dohvata proizvoda, opisati sadržaj poruka i objasniti barem dva različita razloga za isti neuspješan prikaz. Važan je i negativan zaključak: što se na temelju opažanja još ne može znati. Primjer naručivanja nastavit ćemo kroz kolegij, no današnje predavanje obrađuje dohvat, a ne cjelovit sustav narudžbi.

Nakon predavanja moći ćete (u zagradi je razina revidirane Bloomove taksonomije):

- **objasniti** kako preglednik traži podatke i prima odgovor *(razumijevanje)*
- **protumačiti** metodu, status i zaglavlja jednostavne HTTP poruke *(razumijevanje)*
- **skicirati** put zahtjeva od preglednika do baze i natrag *(primjena)*
- **razlikovati** odgovornosti komponenti i mjesta na kojima je stanje *(analiza)*
- **odabrati** dokaz koji razlikuje moguće uzroke kašnjenja ili ispada *(vrednovanje)*

Predznanje: JavaScript, razvoj frontenda i korištenje Firebasea. HTTP učimo od početka.

Za nastavnika: Kako biste objasnili razliku između prikaza u pregledniku i podataka koje treba dohvatiti?

Očekivano obrazloženje: Sučelje se može prikazati prije nego što podaci stignu. Način komunikacije tek uvodimo.

3 Što je Firebase radio za nas?

Vrijeme za nastavu: 4–6 min (2 min).

Firebase SDK (Software Development Kit) ovdje je biblioteka koju aplikacija koristi preko JavaScript funkcija. Kada su studenti dohvaćali ili spremali podatke, nisu morali sami pisati mrežne poruke niti pokretati vlastiti poslužiteljski program. To objašnjava zašto im je poziv funkcije poznat, a HTTP nije. Backend je dio sustava koji obrađuje podatke i pravila izvan korisnikova sučelja; dosad su koristili gotovu uslugu.

Ne tvrdimo da svaki poziv SDK-a proizvodi točno jedan mrežni zahtjev: rezultat može ovisiti o lokalnim podacima, sinkronizaciji i konkretnoj Firebase usluzi. Ne pretpostavljamo ni je li prethodni projekt koristio Firestore ili Realtime Database. Danas ćemo komunikaciju proučiti na vlastitom jednostavnom primjeru, bez oponašanja internog Firebase protokola. Kasnije ćemo čitati poruke javne usluge DummyJSON, a zatim isti princip primijeniti na vlastiti backend.

Poznati koraci: **klik → poziv Firebase biblioteke → podaci → prikaz**.



Kad trebamo podatke s poslužitelja, biblioteka obavlja komunikaciju za nas.

- Naš JavaScript poziva biblioteku u pregledniku.
- Biblioteka komunicira s udaljenom uslugom kada treba podatke s poslužitelja.
- Rezultat koristimo u sučelju.

Backend je program iza sučelja koji obrađuje zahtjeve i podatke.

Izvori: Firebase: SDKs and client libraries.

> **Za nastavnika:** Pozovite studente da opišu dohvat koji su radili s Firebaseom, svojim riječima i bez pogađanja naziva funkcija. Pitajte: izvršava li se sav posao unutar komponente? Povežite lokalni poziv biblioteke s udaljenom uslugom. Ne uvodite TCP i ne tvrdite da Firebase uvijek šalje GET ili koristi isti transport. SDK je biblioteka koju pozivamo u kodu; udaljena usluga je program kojem se ona obraća.

4 Kako je nastao web?

> **Vrijeme za nastavu:** 6–9 min (3 min).

Prije nego što pogledamo poruke, korisno je znati odakle dolaze. U ožujku 1989. Tim Berners-Lee u CERN-u, europskom laboratoriju za fiziku čestica, predložio je sustav povezanih dokumenata kojim bi istraživači lakše dijelili informacije. Do kraja 1990. na računalu NeXT radili su prvi web-poslužitelj i prvi preglednik, a na adresi `info.cern.ch` nalazila se prva web stranica. Berners-Lee je sustav 1991. najavio izvan CERN-a, a 30. travnja 1993. CERN je softver za web (osnovni preglednik, poslužitelj i zajedničku programsku biblioteku) dao svima na besplatnu upotrebu, bez naknade za licencu. To je omogućilo da web postane zajednička infrastruktura, a ne proizvod jedne tvrtke.

Za taj je sustav trebalo nekoliko dijelova: format dokumenta (HTML), program koji prikazuje dokumente (preglednik), program koji ih isporučuje (poslužitelj) i pravila po kojima ta dva programa razgovaraju. Ta pravila zovu se **HTTP** (*HyperText Transfer Protocol*). **Hipertekst** (*hypertext*) je tekst s poveznicama prema drugim dokumentima, **transfer** je prijenos, a **protokol** (*protocol*) skup dogovorenih pravila komunikacije.

Prva inačica, kasnije nazvana HTTP/0.9, bila je vrlo jednostavna. Klijent je otvarao vezu i slao jedan redak, primjerice `GET /hypertext/WWW/TheProject.html`. Poslužitelj je vraćao HTML dokument i zatvarao vezu. Nije bilo statusnih kodova ni zaglavlja: ako nešto nije uspjelo, poslužitelj je vraćao HTML tekst koji je mogao pročitati samo čovjek. HTTP/1.0 (1996.) i HTTP/1.1 (1997.) dodali su statusne kodove, zaglavlja i prijenos različitih vrsta sadržaja. HTTP/2 i HTTP/3 kasnije mijenjaju način prijenosa poruka, ali značenje metoda, statusa i zaglavlja koje danas učimo opisano je zajednički za sve inačice.

Izvori: CERN: A short history of the Web, Berners-Lee: Information Management: A Proposal, W3C: The Original HTTP as defined in 1991, MDN: Evolution of HTTP, RFC 9110.

> **Za nastavnika:** Povijest ispričajte kao razlog za današnje dijelove poruke: statusi i zaglavlja dodani su jer su programi, a ne samo ljudi, trebali razumjeti odgovor. Ne ulazite u razlike HTTP/2 i HTTP/3.

5 Prvi webovi: pogledajmo

> **Vrijeme za nastavu:** 9–12 min (3 min).

Prva web stranica i danas se može otvoriti: CERN ju je 2013. obnovio na adresi info.cern.ch. Stranica je običan tekst s poveznicama; klik na poveznicu traži od poslužitelja novi dokument. Isti sadržaj može se pogledati i u simulatoru ranog tekstualnog preglednika, u kojem se poveznice biraju upisom broja. Na worldwideweb.cern.ch nalazi se rekonstrukcija prvog preglednika iz 2019., izrađena za 30. obljetnicu. Prvi preglednik nije služio samo za čitanje, nego i za uređivanje dokumenata.

Sredinom 1990-ih stranice su dobile slike, boje i okvire. Primjer je sačuvana stranica filma *Space Jam* iz 1996.. I takva stranica je i dalje skup gotovih dokumenata koje poslužitelj šalje pregledniku. Današnja trgovina radi drukčije: preglednik izvršava JavaScript, koji nakon prikaza dohvaća podatke o proizvodima. Upravo zato smo predavanje započeli pitanjem gdje su proizvodi.

Ove stranice su obnovljene ili sačuvane verzije. Ne treba pretpostaviti da su u svakom detalju jednake izvornicima.

Izvori: CERN: info.cern.ch, CERN: Where the web was born, CERN 2019 WorldWideWeb Rebuild.

> **Za nastavnika:** Kliknite nekoliko poveznica na prvoj stranici, zatim pokažite tekstualni simulator. Pitajte što nedostaje u odnosu na današnju trgovinu. Očekivano: podaci koji se dohvaćaju nakon prikaza, prijava, košarica.

6 HTTP: zahtjev i odgovor

> **Vrijeme za nastavu:** 12–15 min (3 min).

Kada otvorimo stranicu, preglednik treba dobiti sadržaj od drugog programa. Preglednik je u toj razmjeni **klijent**, a program koji prima zahtjev i priprema odgovor je **poslužitelj**. Klijent započinje razmjenu porukom: primjerice, traži popis proizvoda. Poslužitelj obrađuje poruku i šalje odgovor s rezultatom.

HTTP određuje oblik i značenje tih poruka, a obje ih strane razumiju. Zahtjev (*request*) putuje od klijenta prema poslužitelju, a odgovor (*response*) natrag prema klijentu. HTTP može prenositi HTML dokumente, slike ili podatke za JavaScript; u našem primjeru podaci su zapisani kao JSON. **JSON** (*JavaScript Object Notation*) je tekstni zapis podataka u obliku JavaScript objekata i nizova, primjerice `[{"id": 7, "name": "Bilježnica"}]`; jer je tekst, mogu ga čitati i programi napisani u drugim jezicima.

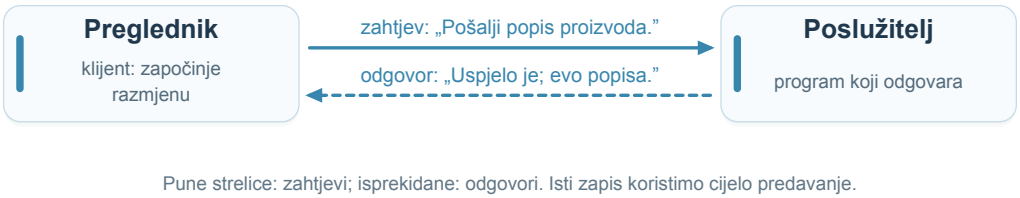


Figure 1: Klijent započinje razmjenu zahtjevom; poslužitelj vraća odgovor.

Za prvi primjer koristimo obične rečenice, ne sintaksu protokola: preglednik traži „Pošalji popis proizvoda”, a poslužitelj odgovara „Uspjelo je; evo popisa”. Uskoro ćemo te dvije rečenice zapisati kao HTTP poruke. Ako razmjena ne uspije, klijent možda neće primiti odgovor; i taj slučaj kasnije razlikujemo od odgovora koji opisuje pogrešku.

Izvori: MDN: Overview of HTTP.

> **Za nastavnika:** Pokažite dvije strelice na slici, jednu za zahtjev i jednu za odgovor; isti zapis vraća se u svim kasnijim dijagramima. Pitajte tko započinje razmjenu. Očekivano: klijent. Ne uvodite još statusne kodove, TLS ni proxy.

7 Prvi pogled u Network

> **Vrijeme za nastavu:** 15–19 min (4 min).

Za ovu demonstraciju ne pokrećemo lokalni poslužitelj. Otvorimo DummyJSON, javnu stranicu s testnim podacima. U pregledniku otvorimo alate za razvoj (*Developer Tools*, tipka F12) i karticu **Network** ili **Mreža**, koja bilježi sve zahtjeve stranice i njihove odgovore„ zatim osvježimo stranicu. Uključimo filtar **All** i odaberimo glavni dokument `dummyjson.com`; u **Response** pogledajmo HTML. Stranica može dohvatiti i skripte, stilove i slike: ne analiziramo svaki redak.

U istoj kartici, uz otvorene alate, otvorimo popis proizvoda. Odaberimo redak `products` i karticu **Response**: sada vidimo JSON. Navigacija može obrisati prethodni popis zahtjeva; usporedbu po potrebi ponovimo povratkom na početnu stranicu. Ne tražimo filtar Fetch/XHR jer se ovdje radi o izravnom otvaranju adrese. Zasad samo pronađemo adresu i odgovor; značenje metode, statusa i zaglavlja uvodimo zatim.

Zadatak: pokažite odgovor koji opisuje stranicu i odgovor koji sadrži podatke. U objektu JSON pronađite polje `products`, a unutar njega `title` i `price` jednog proizvoda. To su testni podaci, ne stvarna trgovina. Prikaz JSON-a u pregledniku nije isto što i gotov katalog: frontend od podataka tek gradi sučelje. Ova dva otvaranja služe usporedbi odgovora; ne tvrdimo da početna stranica nužno automatski traži baš taj endpoint.

Izvori: DummyJSON, Products dokumentacija.

> **Za nastavnika:** Nije potrebna prijava ni instalacija. Neposredno prije sata otvorite oba linka. Ako javna usluga nije dostupna, upotrijebite nastavni backend iz mape `code/01-introduction`: pokrenite `cd code/01-introduction` i `npm start`, otvorite `http://127.0.0.1:3000` i kliknite „Dohvati proizvode”. Lokalni odgovor je jednostavan niz, dok javni primjer ima objekt s poljem products; pokažite tu razliku umjesto pretpostavke da svaki API vraća isti oblik.

8 Adresa i putanja

> **Vrijeme za nastavu:** 19–21 min (2 min).

URL je adresa resursa, primjerice `https://dummyjson.com/products`: `dummyjson.com` govori kojem se odredištu obraćamo, a `/products` koji dio te aplikacije tražimo. Adresa sama još ne govori što s tim resursom želimo učiniti; to će opisati metoda.

Za početak razlikujemo ime odredišta od putanje unutar aplikacije. Resurs je ono čemu pristupamo: ovdje popis proizvoda. Putanja ne mora odgovarati datoteci na disku; poslužiteljski kod može pripremiti podatke na zahtjev. Oznaka https govori da se pristupa uz zaštitu prijenosa, koju ćemo objasniti nakon čitanja poruka.

`https://dummyjson.com/products`

Dio	Uloga
`https`	način pristupa, uz zaštitu prijenosa
`dummyjson.com`	ime odredišta
`/products`	putanja do popisa proizvoda

Izvori: RFC 9110: HTTP Semantics.

> **Za nastavnika:** Koji dio adrese označava odredište, a koji proizvode?
>
> **Očekivano obrazloženje:** dummyjson.com je ime odredišta; /products je putanja. Metodu uvodimo uz prvi zahtjev.

9 Kako pronaći odredište: DNS

> **Vrijeme za nastavu:** 21–23 min (2 min).

Rastavili smo URL; sada povezujemo domensko ime s mrežnom adresom. IP adresa je mrežna oznaka odredišta. Za mrežni poziv preglednik treba pronaći tu adresu prema imenu poput dummyjson.com. DNS je sustav razrješavanja imena; detalje hijerarhije ne obrađujemo na ovom kolegiju. Važno je da odgovor može biti ranije zapamćen i da DNS fazu ne smijemo nacrtati kao obvezan novi udaljeni upit uz svaki HTTP zahtjev.

Pretpostavimo da Network pokazuje grešku razrješavanja imena. Promjena dohvata iz baze tada nije razumna prva intervencija. Najprije provjeravamo adresu i razrješavanje. Suprotno tome, ako backend bilježi primitak našeg zahtjeva, DNS za taj put nije prepreka koja ga je spriječila da stigne. U oba slučaja granicu određujemo dokazom, ne nagađanjem.

Domensko ime čitljiv je naziv, poput `dummyjson.com`. **FQDN** (*Fully Qualified Domain Name*) znači potpuni domenski naziv, od početne oznake do korijena DNS-a. Apsolutni zapis može završiti točkom, primjerice `dummyjson.com.`; u uobičajenom URL-u ta se završna točka izostavlja. FQDN ne uključuje `https://` ni `/products`.

IP adresa mrežna je adresa odredišta. Postoje IPv4 i IPv6 adrese; ovdje ne učimo njihov binarni zapis ni računanje mreža. Jedno ime može imati više adresa, a ista adresa može posluživati više domena.

DNS resolver je usluga koja za naše računalo pronalazi DNS odgovor: koristi spremljeni rezultat ili se obraća drugim DNS poslužiteljima. Dijagram sažima taj postupak, bez prikaza cijele hijerarhije. Nakon pronalaska adrese HTTP razmjena ide prema web odredištu, ne kroz DNS resolver.



- Rezultat može biti spremljen za ponovnu uporabu (predmemorija).
- DNS ne bira funkciju koja obrađuje `/products`.
- Neuspjeh razrješavanja može zaustaviti put prije API-ja.

Domensko ime, IP adresa i putanja različite su stvari.

Izvori: MDN: DNS.

> **Za nastavnika:** Rješava li DNS putanju /products?
>
> **Očekivano obrazloženje:** Ne. Razrješava ime, dok putanju obrađuje HTTP infrastruktura/aplikacija.

10 Zahtjev koji pratimo

> **Vrijeme za nastavu:** 23–25 min (2 min).

Prije cijele poruke pročitajmo `GET /products` običnim riječima: „dohvati proizvode". **GET** je naziv metode kojom tražimo dohvat. **/products** je putanja koju je DummyJSON odabrao za popis proizvoda. Nije ugrađena HTTP naredba niti Firebase funkcija. Zajedno metoda i putanja kažu poslužitelju što tražimo.

Metoda je oznaka radnje koju tražimo; ovdje GET traži dohvat. Zaglavlja su dodatni podaci u obliku naziv: vrijednost. Host navodi ime odredišta. Accept: application/json govori da klijent prihvaća JSON. Tijelo je sadržaj poruke, ako ga ima; ovaj zahtjev nema tijelo. HTTP/1.1 označava verziju protokola u našem čitljivom zapisu. Zapis je nastavno pojednostavljenje, ne potpuna kopija zahtjeva koji je preglednik poslao.

U Network prikazu preglednik može prikazati i zaglavlja koja naš kod nije izričito postavio. Nije potrebno pamtiti sva da bismo pratili zahtjev. Bitno je prepoznati koji dio potječe iz aplikacijskog koda, a koji dodaje platforma. HTTP/2 i HTTP/3 prenose poruke drugačijim uokvirivanjem; tekstualni blok služi čitanju semantike i ne tvrdi da svi protokoli na mreži izgledaju identično.

```
GET /products HTTP/1.1
Host: dummyjson.com
Accept: application/json
```

Metoda • odredište • zaglavlja • moguće tijelo

Ovo je čitljiv zapis **HTTP/1.1**, ne prikaz bajtova HTTP/2 ili HTTP/3.

Izvori: RFC 9110: HTTP Semantics; MDN: Accept.

> **Za nastavnika:** Šalje li se kod naše komponente poslužitelju uz ovaj zahtjev?
>
> **Očekivano obrazloženje:** Ne. Šalje se poruka; backend ne izvršava komponentu.

11 Metoda opisuje namjeru

> **Vrijeme za nastavu:** 25–28 min (3 min).

Danas nam je najvažnije da dohvat ne koristimo kao skriveni poslovni zapis. Ako samo otvaranje poveznice naruči proizvod, različiti klijenti i posrednici mogu izazvati učinak koji korisnik nije tražio. Sigurna metoda znači da klijent ne zahtijeva promjenu stanja ciljanog resursa; uobičajeno bilježenje pristupa time nije isključeno.

Naziv metode ipak nije zaštitna ograda koja automatski ispravlja kod. Programer može pogrešno napisati kod za GET koji mijenja zalihu. Semantiku mora poštovati implementacija. Također, DELETE ne propisuje fizičko brisanje svakog retka: aplikacija definira kako ostvaruje traženi učinak. Detaljne ugovore, idempotentnost i izbor statusa razradit ćemo u trećem tjednu.

Metoda	Namjera u HTTP-u
GET	dohvat podataka o resursu
POST	obrada poslanog sadržaja
PUT	stvaranje ili potpuna zamjena resursa
PATCH	djelomična izmjena
DELETE	uklanjanje resursa iz dostupnog sučelja

`GET /products` **ne smije naručivati proizvod**.

Izvori: MDN: HTTP request methods.

> **Za nastavnika:** Ako promijenimo naziv metode u GET, postaje li operacija automatski sigurna?
>
> **Očekivano obrazloženje:** Ne. Kod i ugovor moraju odgovarati semantici.

12 Odgovor: status, zaglavlja, tijelo

Vrijeme za nastavu: 28–30 min (2 min).

Status je broj kojim poslužitelj opisuje ishod; 200 znači uspjeh. Nakon statusa čitamo zaglavlja, a zatim tijelo poruke. Content-Type je zaglavlje koje navodi format tijela; application/json označava JSON. Prazan redak u čitljivom zapisu odvaja zaglavlja od tijela.

U primjeru odgovora DummyJSON-a 200 označava uspješan HTTP odgovor, a Content-Type najavljuje JSON. Tijelo je reprezentacija koju API namjerno izlaže: ne mora sadržavati sva interna polja baze. Primjerice, prikaz proizvoda ne treba interni nabavni trošak samo zato što ga baza čuva.

Prazan niz u polju products uz status 200 može značiti da nema proizvoda koji odgovaraju upitu. To treba razlikovati od nepostojeće rute ili nemogućnosti dohvata. Uspješan status također ne dokazuje da je frontend uspio ažurirati prikaz. Ako nakon odgovora komponenta očekuje pogrešno ime polja, mrežni dio može biti uspješan, a korisnički rezultat pogrešan. Zato završetak našeg puta uključuje i obradu odgovora u pregledniku.

```
HTTP/1.1 200 OK
Content-Type: application/json

{"products":[{"id":1,"title":"Essence Mascara Lash Princess"}]}
```

Status opisuje ishod zahtjeva. Tijelo nosi podatke.

Prikazana su samo izdvojena polja odgovora DummyJSON-a.

Izvori: MDN: HTTP response status codes; MDN: Content-Type.

> **Za nastavnika:** Može li nakon 200 OK sučelje ostati prazno zbog greške u JavaScriptu?
>
> **Očekivano obrazloženje:** Da. Status nije potvrda uspješnog prikaza.

13 Skupine statusa i granice zaključka

Vrijeme za nastavu: 30–32 min (2 min).

Kodovi imaju dogovorena značenja. Prvo čitamo konkretne primjere 200, 404 i 503, zatim skupine. Zapis 2xx označava kodove koji počinju znamenkom 2, ne doslovan tekst odgovora. Skupine 1xx i 3xx također postoje: informacijski odgovori te preusmjeravanje ili drugo dodatno postupanje. Ne očekujemo učenje svih kodova napamet.

Ako dobijemo 404, neka je komponenta odgovorila; to samo po sebi nije dokaz da cijeli poslužitelj ne radi. Ako odgovor uopće ne stigne, ne izmišljamo HTTP status. Kasnije ćemo razdvojiti prekid komunikacije od odgovora kojim poslužitelj opisuje problem.

Status	Značenje
`200 OK`	zahtjev je uspio
`404 Not Found`	traženi resurs nije pronađen
`503 Service Unavailable`	usluga trenutačno nije dostupna
Nema odgovora	još nemamo HTTP status za tumačenje

Prva znamenka grupira ishode: **2xx uspjeh, 4xx problem zahtjeva, 5xx problem poslužitelja**.

Izvori: MDN: HTTP response status codes.

> **Za nastavnika:** Je li 404 isto što i potpuni izostanak odgovora?
>
> **Očekivano obrazloženje:** Ne. Neka komponenta je vratila HTTP odgovor; treba utvrditi koja.

14 20 HTTP statusa po obiteljima

> **Vrijeme za nastavu:** 32–34 min (2 min).

Odabrani statusi služe kao orijentir, ne rang-lista učestalosti. Prva znamenka opisuje obitelj, a cijeli kod konkretnu poruku.

14.1 1xx — Informacija

Privremena poruka; obrada se nastavlja.

100 Continue, **101 Switching Protocols**.

14.2 2xx — Uspjeh

Zahtjev je prihvaćen ili uspješno obrađen.

200 OK, **201 Created**, **202 Accepted**, **204 No Content**.

14.3 3xx — Preusmjeravanje

Drugo odredište ili uporaba spremljenog sadržaja.

301 Moved Permanently, **302 Found**, **304 Not Modified**, **307 Temporary Redirect**.

14.4 4xx — Problem zahtjeva

Zahtjev nije moguće prihvatiti u ovom obliku ili kontekstu.

400 Bad Request, **401 Unauthorized**, **403 Forbidden**, **404 Not Found**, **409 Conflict**, **429 Too Many Requests**.

14.5 5xx — Problem poslužitelja

Poslužitelj ne može ispuniti zahtjev.

500 Internal Server Error, **502 Bad Gateway**, **503 Service Unavailable**, **504 Gateway Timeout**.

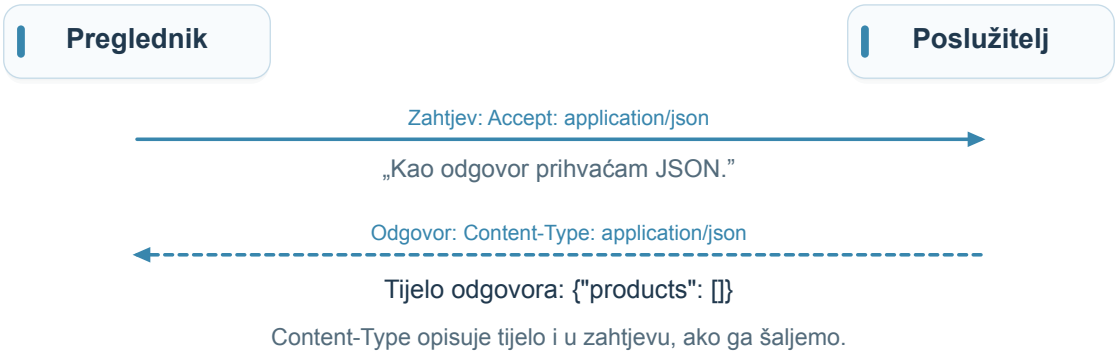
202 ne jamči dovršetak posla; 204 nema tijelo odgovora. 304 upućuje na ranije spremljeni sadržaj. 401 znači da nedostaje valjana provjera identiteta, 403 da poslužitelj odbija pristup. 409 označava sukob s trenutačnim stanjem resursa, 429 previše zahtjeva. Kod 502 posrednik je dobio nevaljan odgovor sljedećeg poslužitelja, kod 504 nije dobio odgovor na vrijeme. Ne izjednačujemo izostanak odgovora s HTTP kodom: tada kod možda uopće nije primljen.

Izvor: MDN: statusi.

15 Accept i Content-Type

Vrijeme za nastavu: 34–36 min (2 min).

Već smo vidjeli zaglavlje u zahtjevu i zaglavlje u odgovoru. Sada ih uspoređujemo kako bismo razumjeli zašto opis formata i stvarni sadržaj moraju odgovarati jedan drugome.



Ove dvije oznake često se zamijene jer u primjerima imaju istu vrijednost. Accept je zahtjev klijenta vezan uz reprezentaciju odgovora. Content-Type opisuje sadržaj konkretne poruke, bilo zahtjeva ili odgovora. Server koji šalje JSON mora stvarno poslati valjan zapis, a ne samo promijeniti zaglavlje.

U kasnijoj lokalnoj demonstraciji namjerno ćemo vratiti HTML umjesto očekivanog JSON-a. Pokušaj čitanja tog sadržaja kao JSON-a tada neće uspjeti. Razlikujemo protokolsku

poruku od JavaScript objekta u memoriji. Serijalizacija stvara tekst za prijenos; parsiranje teksta ponovno stvara vrijednost u drugom procesu. Ta vrijednost nije zajednički objekt s bazom ili backendom.

`Accept: application/json`
Što klijent prihvaća kao odgovor?

`Content-Type: application/json`
Koji format ima ovo tijelo poruke?

Zaglavlje **ne pretvara** HTML ili objekt u JSON.

Pregled deset čestih zaglavlja zahtjeva i deset zaglavlja odgovora nalazi se na kraju poglavlja.

Izvori: MDN: Content-Type; MDN: Accept.

> **Za nastavnika:** Može li Content-Type: application/json učiniti nevaljan tekst valjanim JSON-om?
>
> **Očekivano obrazloženje:** Ne. Zaglavlje opisuje sadržaj, ne preoblikuje ga.

16 Naš nastavni backend

> **Vrijeme za nastavu:** 36–38 min (2 min).

Dosad smo čitali odgovore DummyJSON-a, tuđeg backenda na internetu. Vidimo što vraća, ali ne znamo kako je građen iznutra, pa o njegovoj arhitekturi ništa ne tvrdimo. Od sada koristimo vlastiti nastavni backend: mali Node.js program u mapi `code/01-introduction` materijala kolegija (glavna datoteka `server.mjs`; svako predavanje ima vlastitu kopiju u `code/`). Pokreće se u terminalu naredbama `cd code/01-introduction` i `npm start`; potreban je Node.js 20 ili noviji, bez dodatnih paketa. Program sluša na adresi `http://127.0.0.1:3000`; adresa 127.0.0.1 znači „ovo isto računalo”, pa nastavnikov backend studenti ne mogu otvoriti sa svojih računala, ali mogu pokrenuti vlastitu kopiju iz mape `code/01-introduction`. Isti backend razvijamo kroz cijeli semestar: pravu bazu dodajemo u 4. predavanju, a dotad su podaci simulirani u memoriji.

Naš primjer vraća jednostavan niz s poljima id, name i price, dok DummyJSON vraća objekt s poljem products i nazivom title. Ista putanja `/products` kod dva backenda ima različit odgovor. Daljnji relativni pozivi /products odnose se na vlastiti backend.

Početni model ima preglednik, vlastiti backend i bazu. API (Application Programming Interface) ovdje znači sučelje kojim se drugi programi obraćaju backendu: koje zahtjeve prihvaća i kakve odgovore vraća. Dogovor o putanjama, metodama i obliku podataka zvat ćemo API ugovor. Strelice pokazuju tko koga poziva, ne vrijeme. Posrednika dodajemo tek nakon osnovnog puta.

Broj okvira nije broj fizičkih računala. Preglednik, Node.js i baza mogu se izvršavati na istom prijenosniku ili na različitim računalima. Baza je iz perspektive backenda udaljena

zavisnost kada joj pristupa preko mreže. U ovom modelu frontend ne otvara izravnu vezu prema bazi: API je mjesto preko kojeg dobiva dopuštenu reprezentaciju podataka.

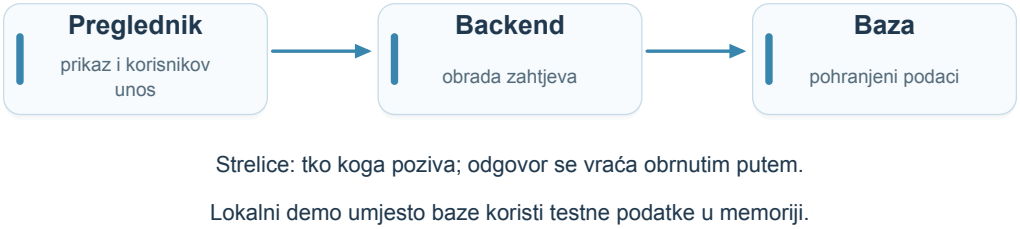


Figure 2: Početni model: preglednik šalje zahtjev backendu, a backend pristupa bazi.

Jedna stranica surađuje s više komponenti.

Preglednik prikazuje podatke; backend provodi pravila; baza čuva zapise.

Izvori: MDN: Overview of HTTP; MDN: Proxy servers and tunneling.

> **Za nastavnika:** Može li jedan stroj sadržavati više prikazanih okvira?
>
> **Očekivano obrazloženje:** Da. Logičke odgovornosti i procesi nisu isto što i fizički strojevi.

17 Utipkajmo HTTP zahtjev ručno

> **Vrijeme za nastavu:** 38–42 min (4 min).

HTTP/1.1 poruka je običan tekst, pa je možemo napisati i bez preglednika. Pokrenimo nastavni backend (`cd code/01-introduction`, zatim `npm start`), a zatim u drugom terminalu otvorimo vezu prema njemu programom koji samo prenosi utipkane znakove:

Sustav	Naredba	Napomena
macOS	`nc -c 127.0.0.1 3000`	`nc` je ugrađen; `-c` završava retke s CRLF
Linux	`nc -C 127.0.0.1 3000`	inačica netcat-openbsd ili `ncat`; inače `telnet 127.0.0.1 3000`
Windows	`telnet 127.0.0.1 3000`	prvo uključiti značajku Telnet Client

HTTP zahtijeva da retci završavaju znakovima CR i LF (`\r\n`). Telnet ih šalje sam, a `nc` tek uz navedenu opciju. Kada se veza otvori, utipkamo:

```
GET /products HTTP/1.1
Host: 127.0.0.1
```

Prvi redak sadrži metodu, putanju i inačicu protokola. Drugi je zaglavlje `Host`, koje je u HTTP/1.1 obavezno. Prazan redak označava kraj zaglavlja; dok ga ne pošaljemo, poslužitelj čeka nastavak. Odgovor počinje statusnim retkom `HTTP/1.1 200 OK`, slijede zaglavlja poput `Content-Type: application/json` i, nakon praznog retka, tijelo s proizvodima. Ako odgovor ima zaglavlje `Transfer-Encoding: chunked`, tijelo se šalje u dijelovima, a heksadecimalni brojevi oko JSON-a su duljine dijelova, ne podaci.

Zatim isprobamo dvije pogreške. Bez retka `Host` naš Node.js poslužitelj vraća `400 Bad Request`, jer zahtjev nije ispravan HTTP/1.1. Za `GET /nesto` vraća `404 Not Found` i JSON s porukom pogreške. U sva tri slučaja veza je uspjela; razlikuje se samo značenje odgovora.

Isto rade preglednik i `fetch`: oblikuju ovakvu poruku i čitaju odgovor umjesto nas. Network u alatima za razvoj prikazuje te iste dijelove. Na HTTPS vezi ovo ne možemo utipkati jer se promet šifrira; to objašnjavamo kod HTTPS-a.

Izvori: RFC 9112: HTTP/1.1, RFC 9110: HTTP Semantics.

> **Za nastavnika:** Demonstraciju pripremite unaprijed u dva terminala. Na Windowsu telnet ne prikazuje utipkane znakove: Ctrl+], `set localecho`, Enter. Na macOS-u telnet više nije ugrađen, zato koristimo `nc`. Linux i Windows naredbe nisu isprobane na nastavnom računalu; provjerite ih prije sata.

18 Veza i port

> **Vrijeme za nastavu:** 42–44 min (2 min).

Naredbom `nc 127.0.0.1 3000` u prethodnoj demonstraciji otvorili smo vezu prema backendu. Prvi broj je IP adresa, a drugi port. Port je broj koji razlikuje usluge na mrežnom odredištu. localhost i adresa 127.0.0.1 upućuju na vlastito računalo; 3000 odabire naš poslužiteljski program, koji sluša na tom portu. HTTP opisuje značenje poruka, a prijenosni protokol brine o njihovu putovanju.

TCP uspostava veze prethodi razmjeni u našem odabranom modelu. TCP je protokol prijenosa koji uspostavlja vezu i osigurava uređen tok bajtova. Ne pretpostavljamo da studenti poznaju postupak uspostave; dovoljno je objasniti da obje strane najprije pripreme vezu prije razmjene podataka. Pouzdan prijenos toka bajtova ne znači da je poslovna operacija uspješno izvršena. Proces može primiti poruku pa pogrešno obraditi podatke ili se srušiti prije odgovora.

Port povezuje odredišnu komunikaciju s uslugom koja sluša. Otvoren port nije dokaz valjane autorizacije, dobre baze ni točnog rezultata. Za lokalni demo koristimo jedan eksplicitni port i jednu instancu procesa. Kasnije će proxy i balansiranje opterećenja pokazati zašto javna veza i backend proces ne moraju biti jedan-na-jedan.

Za HTTP/1.1 i HTTP/2 tipično pratimo **TCP vezu**.

- IP i port vode do mrežnog odredišta.
- TCP uspostavlja vezu i prenosi tok bajtova.
- Aplikacija mora obraditi primljenu poruku.

HTTP/3 koristi QUIC nad UDP-om; današnji slijed modelira TCP put.

Izvori: MDN: TCP; MDN: Overview of HTTP.

> **Za nastavnika:** Je li uspostavljena TCP veza potvrda da proizvodi postoje?
>
> **Očekivano obrazloženje:** Ne. Omogućuje prijenos, a sadržaj i ishod utvrđuje aplikacija.

19 Program, proces i računalo

> **Vrijeme za nastavu:** 44–47 min (3 min).

Node.js je okruženje u kojem JavaScript možemo pokrenuti izvan preglednika. Njime pokrećemo naš backend. Pritom razlikujemo tri pojma koja se u govoru često miješaju.

Program je datoteka s kodom na disku, kod nas `server.mjs`. Dok ga ne pokrenemo, ne prima zahtjeve i ništa ne obrađuje. **Proces** je jedno pokretanje programa: operacijski sustav mu dodjeljuje radnu memoriju i vrijeme procesora. Naredbom `npm start` u mapi `code/01-introduction` nastaje jedan proces koji sluša na portu 3000. Ako u drugom terminalu pokrenemo `PORT=3001 npm start`, iz istog programa nastaje drugi proces, na portu 3001. **Računalo** istodobno izvršava mnogo procesa: osim naša dva backend procesa, i preglednik i `nc` su procesi.

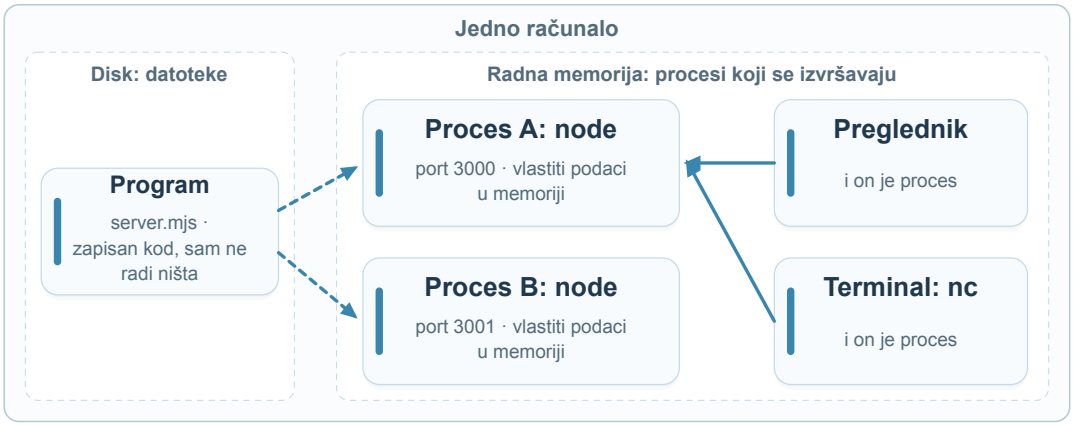


Figure 3: Jedno računalo: program `server.mjs` na disku pokrenut je dvaput, pa u radnoj memoriji postoje dva procesa, A na portu 3000 i B na portu 3001, svaki s vlastitim podacima. Preglednik i nc također su procesi. Zahtjev na port 3000 stiže samo procesu A.

Svaki proces ima vlastitu memoriju. Varijabla koju proces A promijeni ne mijenja se u procesu B, iako su oba nastala iz iste datoteke. Ako jedan proces ugasimo, drugi i dalje odgovara, a datoteka na disku ostaje ista. Isto vrijedi i za druge sudionike: gašenje backend procesa ne gasi preglednik ni bazu.

Riječ **poslužitelj** može značiti računalo ili program. U ovom predavanju njome označavamo ulogu: proces koji sluša na portu i odgovara na zahtjeve.

Pojam	**U našem primjeru**
Program	`server.mjs` na disku
Proces	jedno pokretanje: `npm start`
Poslužitelj	uloga procesa koji sluša na portu i odgovara
Računalo	stroj koji istodobno izvršava mnogo procesa

Procese možemo vidjeti: na macOS-u u Monitoru aktivnosti ili naredbom `ps aux | grep node`, na Windowsu u Upravitelju zadataka, a na Linuxu naredbom `ps aux | grep node`.

> **Za nastavnika:** Pokrenite oba procesa uživo i otvorite obje adrese. Na Windowsu u PowerShellu: `$env:PORT=3001; npm start`. Ako pokrenemo backend dvaput, dijele li procesi automatski istu JavaScript varijablu? Očekivano: ne; za zajedničko stanje potreban je izričit mehanizam, primjerice baza.

20 Handler je granica obrade

Vrijeme za nastavu: 47–49 min (2 min).

Handler je funkcija koja obrađuje primljeni zahtjev. Ruta povezuje metodu i putanju s tom funkcijom: primjerice GET /products odabire funkciju listProducts. Framework može nuditi različitu sintaksu za taj odabir, ali odgovornost ostaje ista.

```
// Pojednostavljeni prikaz odgovornosti, nije framework API.
async function listProducts() {
  const products = await readProducts();
  return { status: 200, body: products };
}
```

readProducts je naša funkcija za dohvat; nije ugrađena JavaScript naredba. U stvarnoj aplikaciji može pristupati bazi, dok lokalna demonstracija koristi unaprijed pripremljen niz. Prikazani objekt opisuje zamišljeni odgovor; izvršivi demo mora posebno postaviti status, zaglavlja i poslati JSON. Čekanje uz await ovdje znači da se rezultat dohvata koristi tek kada je spreman. Podsjetimo na sintaksu prema potrebi, bez pretpostavke iskustva s backendom.

> **Za nastavnika:** Što bira funkciju, što dohvaća podatke, a što određuje ishod? Očekivano: ruta, readProducts i handler. Detalje organizacije koda razvijamo u drugom predavanju.

21 Okvir stranice i podaci stižu odvojeno

Vrijeme za nastavu: 49–52 min (3 min).

Stranica koju korisnik vidi kao jednu cjelinu stiže u dva odvojena zahtjeva. Prvo preglednik dohvaća samu aplikaciju: HTML dokument, JavaScript i stilove. Od toga nastaje okvir stranice, primjerice zaglavlje, izbornik i gumbi, ali još bez proizvoda. Zatim JavaScript šalje zaseban zahtjev `GET /products`, prima JSON i od njega gradi kartice proizvoda. Zato je na uvodnom primjeru stranica bila otvorena, a proizvoda nije bilo: prvi korak je uspio, a za drugi tek treba provjeriti što se dogodilo.

Za naš primjer izričito biramo frontend koji nakon učitavanja traži JSON. Nije svaka web aplikacija tako organizirana: poslužitelj može i unaprijed pripremiti HTML s podacima. Odabrani model odgovara poznatoj Vue aplikaciji i pomaže odvojiti dvije faze koje korisnik vidi kao jednu stranicu.

U Network kartici zato treba razlikovati dokument, JavaScript resurse i fetch/XHR poziv za proizvode. Ako je dokument već u predmemoriji, početni prikaz može uspjeti i kada API kasnije nije dostupan. Ne dokazujemo da se to dogodilo samo zato što postoji predmemorija; provjeravamo konkretan zapis zahtjeva. Vizualna sličnost rezultata nije dokaz istog uzroka.

Korak	Zahtjev	Rezultat
1. Učitavanje aplikacije	`GET /` (HTML; u većim aplikacijama i `.js`, `.css`)	okvir stranice: zaglavlje, izbornik, gumbi
2. Dohvat podataka	`GET /products`, šalje ga JavaScript	JSON s proizvodima
3. Prikaz	nema novog zahtjeva	JavaScript od JSON-a gradi kartice proizvoda

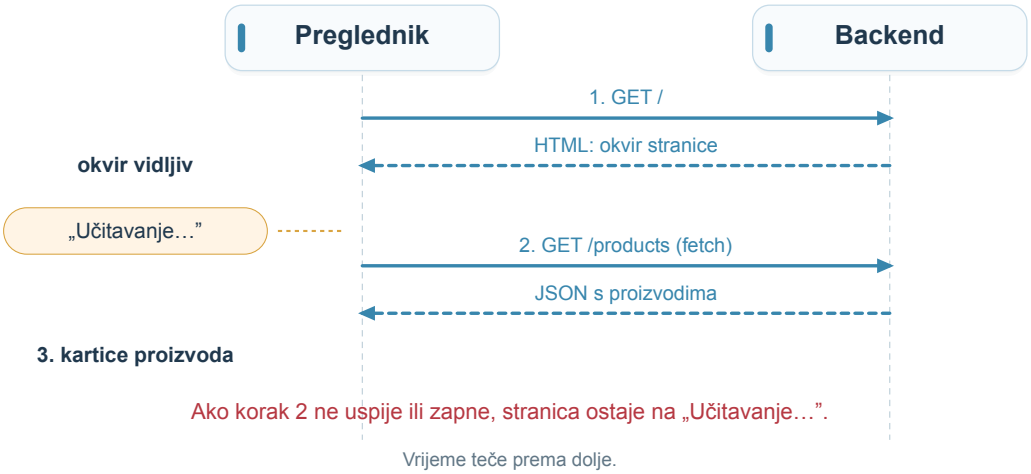


Figure 4: Okvir stranice i podaci stižu u dva zahtjeva; između njih stranica prikazuje „Učitavanje…”.

Naš nastavni `index.html` ima ugrađenu skriptu, pa u Networku na našem backendu vidimo samo `GET /` i `GET /products`.

Korak 1 može uspjeti, a korak 2 ne: okvir stranice je vidljiv, proizvoda nema.

Izvori: MDN: Overview of HTTP.

> **Za nastavnika:** Koji Network redak govori o proizvodima: dokument (/) ili /products?
>
> **Očekivano obrazloženje:** /products. Uspjeh dokumenta sam ne rješava pitanje API-ja.

22 Naivan dohvat proizvoda

> **Vrijeme za nastavu:** 52–56 min (4 min).

Dosad smo adresu otvarali u pregledniku; sada isti dohvat pokrećemo iz JavaScripta. Ugrađeni fetch šalje HTTP zahtjev; bez dodatnih postavki koristi GET. Relativna putanja /products odnosi se na odredište naše lokalne stranice, ne na DummyJSON. await čeka rezultat asinkrone operacije. response sadrži odgovor, a response.json() čita njegovo tijelo kao JSON. U primjerima su products, loading i error obične varijable stanja unutar asinkrone funkcije; njihovo povezivanje s prikazom izostavljamo.

Ovaj kratki kod namjerno nije dovršeno rješenje. Pretpostavlja da se poziv uspješno završio, da je tijelo JSON, da njegov oblik odgovara prikazu te da korisnik ne treba informaciju o čekanju. Te pretpostavke ispitujemo pojedinačno. HTTP odgovor s kodom 503 nije isto što i neuspjeh mrežnog poziva: fetch može vratiti Response i za neuspješan HTTP status.

Razmotrimo tri ulaza. Za 200 i praznu listu dohvat može biti potpuno uspješan. Za 503 i JSON opis greške kod može pogrešan objekt pridružiti listi proizvoda. Za HTML stranicu greške pretvorba u JSON može propasti. Prije ispravka tražimo od studenata da predvide opažanje za svaki slučaj. Problem nije u nedovoljnom broju redaka, nego u neizrečenom ugovoru.

```
const response = await fetch('/products');
products = await response.json();
```

Što prikazujemo tijekom čekanja, za `503` ili za `[]`?

Izvori: MDN: Using the Fetch API.

> **Za nastavnika:** Hoće li fetch automatski baciti iznimku za svaki HTTP 404?
>
> **Očekivano obrazloženje:** Ne. Treba provjeriti status/response.ok.

23 Razlikuj stanja prikaza

> **Vrijeme za nastavu:** 56–59 min (3 min).

Ispravak počinje modelom stanja, a ne novom bibliotekom. Prije slanja postavimo oznaku učitavanja. Nakon uspješnog odgovora s očekivanim oblikom podataka odlučujemo između popisa i praznog rezultata. Neuspjeh čuvamo odvojeno da ga korisnik ne zamijeni s tvrdnjom da proizvodi ne postoje. U završnoj grani uklanjamo oznaku čekanja.

Ako već postoji stari popis, dizajn može odlučiti zadržati ga uz obavijest da osvježavanje nije uspjelo. Drugi je izbor ukloniti prikaz. Oba zahtijevaju jasnu oznaku aktualnosti. U našem jednostavnom primjeru uklanjamo stare podatke pri novom pokušaju kako bismo bez dodatnih stanja pratili jedan zahtjev. To je nastavna odluka, ne univerzalno pravilo sučelja.

Stanje	Poruka korisniku
Čekanje	Učitavanje proizvoda…
Uspjeh, postoje podaci	Popis proizvoda
Uspjeh, prazna lista	Nema proizvoda za ovaj upit
Neuspjeh	Dohvat nije uspio

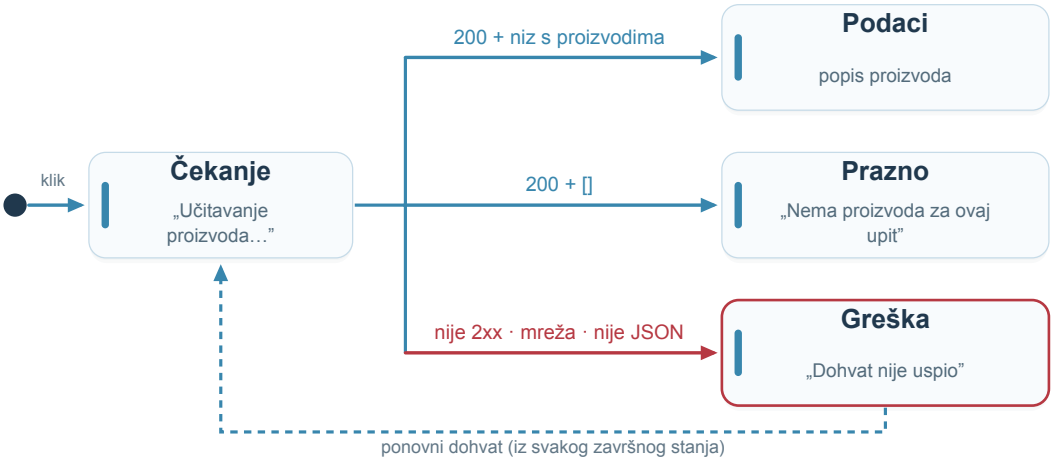


Figure 5: Stanja prikaza kao dijagram stanja: iz čekanja odgovor vodi u podatke, prazan popis ili grešku; ponovni dohvat vraća u čekanje.

Prazna lista nije zamjena za grešku.

> **Za nastavnika:** Je li return [] u svakom catch bloku dobra obrada greške?
>
> **Očekivano obrazloženje:** Ne: pretvara neuspjeh u privid uspješnog praznog odgovora.

24 Dohvat s eksplicitnom provjerom

> **Vrijeme za nastavu:** 59–63 min (4 min).

Ispravak sada slijedi odmah nakon modela stanja. r.ok je true za uspješan status od 200 do 299. Za ostale statuse izričito bacamo grešku. JSON može imati neočekivan oblik pa provjeravamo i da smo dobili niz; stvarni demo dodatno provjerava polja svakog proizvoda. catch bilježi neuspjeh, a finally uklanja oznaku čekanja u oba ishoda.

Kod sada razdvaja čekanje, uspjeh i neuspjeh te ne smatra svaki Response uspjehom. Array.isArray provjerava samo vanjski oblik; to nije potpuna validacija svakog proizvoda. Ako

ugovor zahtijeva brojčani id i tekstualni name, potrebne su i te provjere. Skraćeni prikaz namjerno ističe mjesto odgovornosti bez uvođenja biblioteke za sheme.

Također još ne rješava dva preklopljena zahtjeva koja se dovrše obrnutim redom. U demonstraciji onemogućujemo novo slanje dok traje prethodno kako bismo istražili jedan tok. Za složenije sučelje trebalo bi odlučiti kako se otkazuju ili ignoriraju zastarjeli rezultati. Uklanjanje spinnera u finally važno je i nakon neuspjeha: čekanje ne smije ostati jedino stanje koje korisnik vidi.

```
loading = true;
error = null; products = [];
try {
  const r = await fetch('/products');
  if (!r.ok) throw new Error(`HTTP ${r.status}`);
  const data = await r.json();
  if (!Array.isArray(data)) throw new Error('Oblik podataka');
  products = data;
} catch (e) {
  error = 'Dohvat nije uspio';
} finally {
  loading = false;
}
```

Dio asinkrone funkcije; `products`, `loading` i `error` su stanje prikaza.

Izvori: MDN: Using the Fetch API.

> **Za nastavnika:** Koje pretpostavke ovaj ispravak još nije uklonio?
>
> **Očekivano obrazloženje:** Oblik svakog elementa, konkurentni pozivi i aktualnost podataka zahtijevaju dodatni dizajn.

25 Isti zahtjev, četiri ishoda

> **Vrijeme za nastavu:** 63–67 min (4 min).

Demonstracija koristi naš backend iz mape `code/01-introduction` (`npm start`, zatim `http://127.0.0.1:3000`). Stranica ima gumbe koji šalju isti zahtjev `GET /products`, uz parametar `case` kojim backend bira simulirani ishod. Taj parametar je naš dogovor za demonstraciju, nije dio HTTP-a. Odgovor backenda za svaki gumb je zadan. Predviđamo ono što iz njega slijedi na stranici: stranica koristi kod iz prethodnog odjeljka, pa za svaki odgovor pratimo je li `r.ok` istinit, uspijeva li `r.json()` i je li rezultat niz. Uspjeh daje podatke, prazan niz daje prazno stanje (to nije greška), `503` daje grešku jer `r.ok` nije istinit, a spor odgovor dvije sekunde prikazuje učitavanje pa podatke. Peti gumb, „Neočekivan HTML”, vraća `200`, ali `r.json()` ne uspijeva, pa je stanje greška iako je status uspješan. U demonstraciji baza je izričito simulirana memorijskim podacima; nema stvarnog PostgreSQL-a ni

MongoDB-a, TLS-a ni reverse proxyja. Cilj je pregled HTTP poruke i ponašanja klijenta, a ne dokaz svojstava produkcijske arhitekture.

Odaberite uspjeh, praznu listu, namjerni 503 i odgodu. Usporedite konzolni zapis servera i Network. Zatim, ako vrijeme dopušta, zaustavite proces i ponovite dohvat: HTTP 503 koji server namjerno pošalje i nedostupan proces nisu ista pojava. Ako lokalni server nije moguće pokrenuti, koristite ispisane primjerene poruke iz prethodnih slajdova; ne predstavljajte ih kao opažena mjerenja.

Svaki gumb na našem backendu šalje `GET /products`; backend namjerno odgovara drukčije. Stranica koristi kod s prethodnog slajda.

Gumb	**Odgovor backenda**	**Stanje prikaza?**
Dohvati proizvode	`200` i proizvodi	?
Prazno	`200` i `[]`	?
Pogreška poslužitelja	`503`	?
Sporo	nakon 2 s: `200` i proizvodi	?

Prema kodu predvidite: učitavanje, podaci, prazno ili greška? Zatim kliknemo i provjerimo.

Izvori: Node.js: HTTP API; MDN: Using the Fetch API.

> **Za nastavnika:** Što je različito između namjernog 503 i ugašenog procesa?
>
> **Očekivano obrazloženje:** U prvom slučaju postoji HTTP odgovor, u drugom se ne mora dobiti nikakav odgovor.

26 HTTPS i granica zaštite

> **Vrijeme za nastavu:** 67–70 min (3 min).

Dosad je cijela demonstracija radila na jednom računalu. Na internetu poruka između preglednika i poslužitelja prolazi mrežama kojima ne upravljamo: HTTP tekst kakav smo ručno utipkali mogao bi pročitati ili izmijeniti svatko na tom putu. HTTPS koristi TLS radi zaštite podataka u prijenosu i provjere identiteta poslužitelja. Ne poistovjećujemo šifriranu komunikaciju s ispravnom aplikacijom. Ako korisnik preko savršeno zaštićene veze zatraži tuđi zapis, aplikacija i dalje mora provjeriti ovlasti.

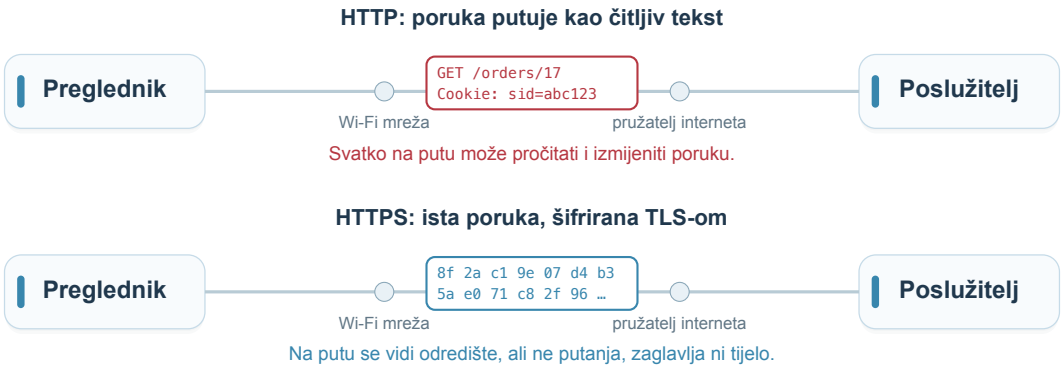


Figure 6: Ista poruka na putu: preko HTTP-a čitljiva, preko HTTPS-a šifrirana. Odredište je i dalje vidljivo.

Slika pokazuje što TLS mijenja na putu. Bez njega svaki uređaj kroz koji poruka prolazi, primjerice Wi-Fi mreža u kafiću ili pružatelj interneta, vidi putanju, zaglavlja (i cookie) te tijelo. S TLS-om oni vide da se spajamo na određeno odredište, ali ne i sadržaj poruke.

Certifikat je digitalna potvrda kojom preglednik provjerava identitet poslužitelja. Za početnu sliku promatramo zaštićenu komunikaciju preglednika s poslužiteljem. Uskoro ćemo dodati posrednika i vidjeti da zaštita jednog odsječka ne opisuje nužno ostatak puta. Pitanje za svaku strelicu glasi: tko su krajnje točke, što je zaštićeno i koji sudionik poslije vidi podatke u čitljivom obliku?

TLS štiti komunikaciju između svojih krajnjih točaka.

Preglednik provjerava identitet poslužitelja putem certifikata.

TLS ne provjerava:

- smije li korisnik čitati tuđu narudžbu
- je li cijena u zahtjevu ispravna
- postoji li greška u backend kodu

Izvori: MDN: Transport Layer Security.

> **Za nastavnika:** Provjerava li HTTPS smije li korisnik vidjeti tuđu narudžbu?
>
> **Očekivano obrazloženje:** Ne. To mora provjeriti aplikacija koja obrađuje zahtjev.

27 Granica povjerenja

> **Vrijeme za nastavu:** 70–72 min (2 min).

TLS štiti poruku na putu, ali ne provjerava ono što je klijent u nju upisao. Naša aplikacija zasad samo čita proizvode, ali već možemo provjeriti gdje bi kasnije pripadala pravila naručivanja. Zamislimo polje cijene prikazano u komponenti. Korisnik može izmijeniti vlastiti prikaz ili poslati zaseban HTTP zahtjev. Backend ne smije cijenu iz takvog zahtjeva smatrati službenom samo zato što ju je poslala „naša aplikacija".

Razlika je između pomoći pri unosu i autoritativne odluke. Frontend može rano objasniti grešku i poboljšati iskustvo. Backend mora provjeriti uvjete operacije, a baza može osigurati točno definirana ograničenja. Ne pripisujemo bazi sva poslovna pravila automatski: treba znati koja su stvarno ugrađena. Ovu granicu ponovno ćemo razraditi kod autorizacije i sigurnosti.

Klijent šalje podatke koje može izmijeniti.

- Preglednik pomaže korisniku ispuniti zahtjev.
- Backend odlučuje što je dopušteno.
- Baza provodi definirana ograničenja podataka.

Skriven gumb ne sprječava izravan API zahtjev.

> **Za nastavnika:** Ako se gumb Obriši ne prikazuje, može li zahtjev ipak stići?
>
> **Očekivano obrazloženje:** Da. Potrebna je provjera dopuštenja u backendu; danas samo lociramo odgovornost.

28 Webshop: tko određuje cijenu?

> **Vrijeme za nastavu:** 72–74 min (2 min).



Promjena prikaza sama ne mijenja bazu. **Problem nastaje ako backend povjeruje poslanom iznosu.**

Pitajte: je li skriveno ili onemogućeno polje cijene zaštita? Nije. Korisnik kontrolira vlastiti klijent i može zaobići sučelje. U lošem primjeru poslana cijena postaje cijena narudžbe. U dobrom backend prihvaća identifikator proizvoda i količinu, provjerava da proizvod postoji,

količina je dopuštena i korisnik smije naručiti, pa primjenjuje važeću cijenu i eventualne odobrene popuste. Dodatnu cijenu iz zahtjeva može odbiti ili zanemariti. Ovo nije potpuna implementacija naplate; zalihu i istodobne narudžbe obrađujemo kasnije.

Izvor: OWASP: Input Validation.

29 Reverse proxy je posrednik

Vrijeme za nastavu: 74–77 min (3 min).

Sada početnom modelu dodajemo posrednički program između preglednika i backenda. To nije obvezan dio lokalne demonstracije. Primjer 504 znači da posrednik nije na vrijeme dobio odgovor od sljedećeg poslužitelja; 502 označava nevaljan odgovor koji je od njega dobio. Kada kažemo zavisnost, mislimo na komponentu čiji rezultat druga komponenta treba, poput baze koju backend poziva.

Reverse proxy stoji ispred poslužitelja aplikacije iz perspektive klijenta. U našem modelu prihvaća javni zahtjev i otvara ili ponovno koristi zasebnu vezu prema backendu. To uvodi još jednu granicu kašnjenja i mogućeg neuspjeha, ali omogućuje zajednički ulaz za više procesa (instanci) istog backenda.

Pretpostavimo da proxy prekine čekanje, dok backend još obrađuje upit. Klijent može dobiti pogrešku od proxyja, a backend naknadno završiti obradu. Takva situacija nije dokaz da je baza prazna. Za dijagnozu treba povezati zapis na javnom ulazu i zapis u backendu. Ne obećavamo automatsko otkazivanje obrade samo zato što je jedan sudionik prestao čekati.

Javni ulaz nije nužno backend proces.

Proxy može:

- završavati TLS i prosljeđivati zahtjeve
- usmjeravati prema jednom od backend procesa
- vratiti vlastitu pogrešku ako backend ne odgovori

502/504 prvo povezujemo s komponentom koja ih je vratila.

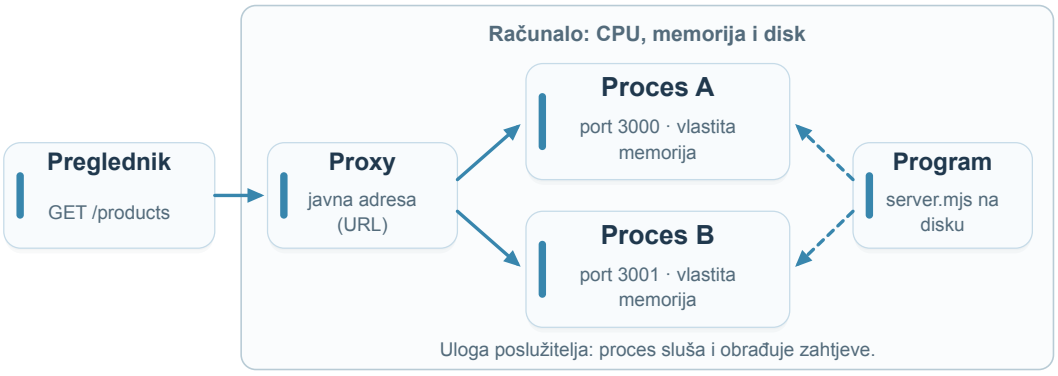
Izvori: MDN: Proxy servers and tunneling.

Za nastavnika: Ako je proxy dostupan, mora li i backend biti dostupan?

Očekivano obrazloženje: Ne. Sudionici mogu imati različite ishode i kvarove.

29.1 Jedan URL, više procesa

Sada možemo spojiti proxy s dva procesa iz ranije demonstracije. Preglednik zna samo javnu adresu. Proxy prosljeđuje zahtjev procesu A ili procesu B, a preglednik ne zna koji je odgovorio. Budući da procesi ne dijele memoriju, stanje koje mora vrijediti za sve zahtjeve ne smije živjeti samo u memoriji jednog procesa.



Pune strelice: zahtjev. Isprekidane: pokretanje programa (npm start).

Isti URL može doći do bilo kojeg procesa; procesi ne dijele memoriju.

Figure 7: Isti program `server.mjs` pokrenut je dvaput na istom računalu. Proxy na javnoj adresi prosljeđuje zahtjev procesu A ili procesu B; svaki proces ima vlastitu memoriju. Pune strelice označavaju zahtjev, isprekidane pokretanje programa.

URL nije identifikator jednog procesa.

30 Slijed 1: preglednik šalje zahtjev

Vrijeme za nastavu: 78–80 min (2 min).

Sada kartu zavisnosti pretvaramo u vremenski slijed. Početni događaj je korisnički klik ili učitavanje prikaza. Kod određuje URL i pokreće poziv. Preglednik ima lokalnu oznaku čekanja; backend u ovom trenutku možda još nije vidio nijedan bajt. Puna strelica označava poziv, a vrijeme na slici teče prema dolje.

Slijed prikazuje produkcijski model s proxyjem i bazom. Naš lokalni backend nema proxy, a podatke drži u memoriji; uloge ostaju iste. Radi preglednosti pretpostavljamo da je potrebna komunikacijska veza uspostavljena i da odgovor ne dolazi iz predmemorije (*cache*), odnosno spremljene kopije ranijeg odgovora. Te pretpostavke izgovaramo jer stvarni Network zapis može izgledati drugačije. Učitavanje sučelja nije „dokaz da server radi”; to je lokalna informacija da je naša funkcija započela posao.

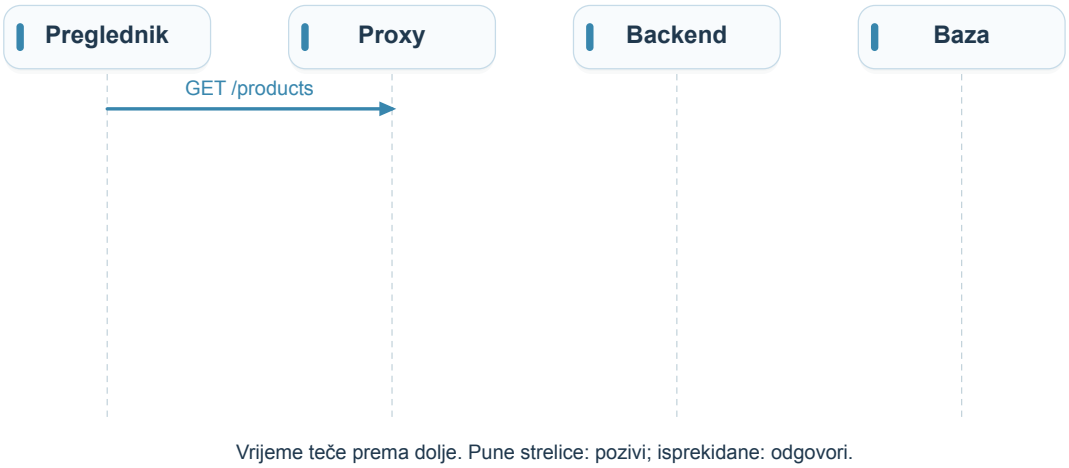


Figure 8: Prvi korak: preglednik šalje GET /products proxyju. Vrijeme teče prema dolje.

JavaScript funkcija pokreće `GET /products`.

Prije slanja: `loading = true`.

Pretpostavke: veza je dostupna; zanemarujemo predmemoriju; jedan zahtjev.

> **Za nastavnika:** Koji sudionik trenutačno zna da je korisnik kliknuo?
>
> **Očekivano obrazloženje:** Preglednik; ostali tek nakon primitka odgovarajuće poruke.

31 Slijed 2: backend traži podatke

> **Vrijeme za nastavu:** 80–82 min (2 min).

Nakon prosljeđivanja backend utvrđuje koja funkcija obrađuje metodu i putanju. U našem nastavnom modelu ona provjerava parametar upita i poziva funkciju za dohvat proizvoda. Detalje SQL-a ili MongoDB upita razradit ćemo u četvrtom tjednu; danas nas zanima da taj poziv ima početak, trajanje, rezultat ili grešku.

Backend može čekati dostupnu vezu prema bazi, izvršavanje upita ili prijenos rezultata. Ništa od toga preglednik ne vidi kao gotov proizvod sve dok odgovor ne bude oblikovan i prenesen. Zabilježimo zato gdje postoje privremene kopije i gdje je trenutno mjerljivo trajanje. Riječ await u backend kodu nije udaljeni poziv iz preglednika: to je lokalno čekanje rezultata određene asinkrone operacije.

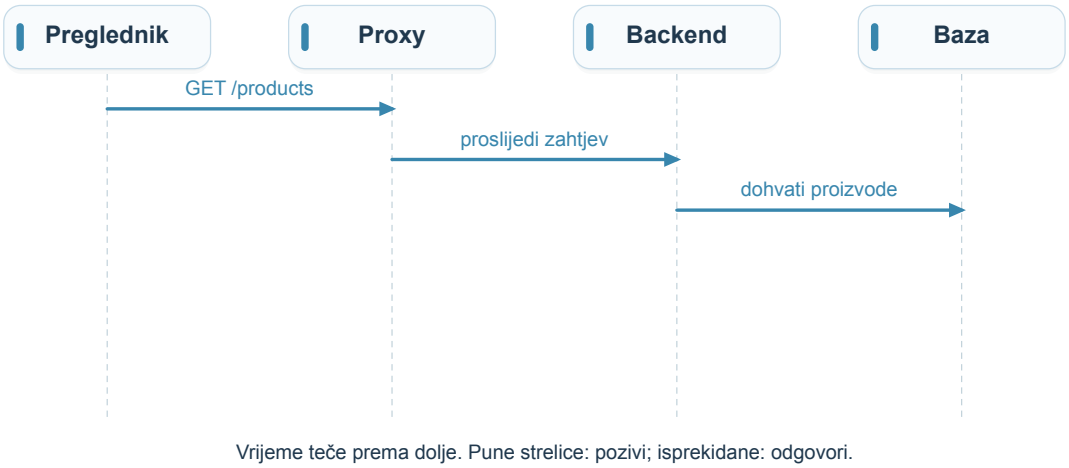


Figure 9: Proxy prosljeđuje zahtjev backendu, a backend traži proizvode iz baze.

Proxy prosljeđuje poruku.

Backend odabire handler i traži proizvode iz baze.

Čekanje baze nije isto što i slanje odgovora.

> **Za nastavnika:** Gdje biste mjerili vrijeme upita?
>
> **Očekivano obrazloženje:** Oko poziva prema bazi u backendu; odvojeno od ukupnog vremena klijenta.

32 Slijed 3: backend oblikuje odgovor

> **Vrijeme za nastavu:** 82–84 min (2 min).

Rezultat baze još nije HTTP odgovor. Backend ga može presložiti, ograničiti na potrebna polja i pretvoriti u JSON. U našem primjeru klijentu trebaju id, name i price. Interni zapis može sadržavati dodatne informacije koje ne treba izložiti. To je odluka aplikacijskog ugovora.

Ako čitanje uspije, backend postavlja odgovarajući status i format tijela. Ako čitanje ne uspije, ne smije tiho vratiti prazan niz kao da je dohvatio nula proizvoda. Razlika između „nema rezultata” i „nismo mogli saznati rezultat” ključna je za ispravno sučelje i kasniju dijagnostiku. Današnji primjer ne uvodi stvarnu bazu u izvršivi demo: tamo je ta granica označena simuliranom funkcijom.

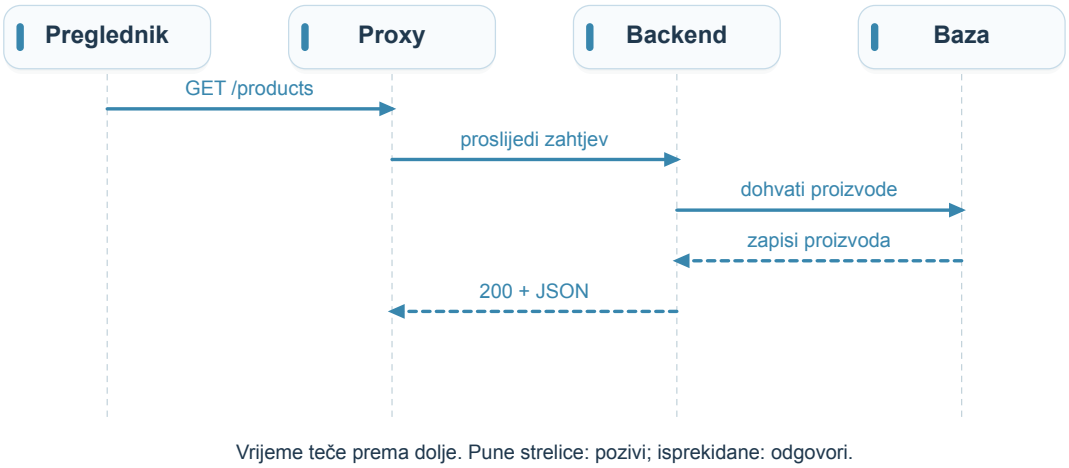


Figure 10: Baza vraća zapise backendu; backend šalje HTTP odgovor proxyju.

Baza vraća zapise.

Backend odabire javna polja i pretvara podatke u JSON.

Odgovor može sadržavati samo dio podataka iz baze.

> **Za nastavnika:** Smije li frontend dobiti svako polje retka samo zato što postoji?
>
> **Očekivano obrazloženje:** Ne. API izlaže odabranu reprezentaciju.

33 Slijed 4: prikaz se ažurira

> **Vrijeme za nastavu:** 84–86 min (2 min).

Odgovor prelazi posljednju mrežnu granicu, ali korisnički posao još nije dovršen. Frontend provjerava status, čita tijelo i tumači njegov oblik. Zatim mijenja podatke na koje je vezan prikaz. Ako umjesto niza očekuje objekt s poljem items, a ugovor vraća niz, potrebno je uskladiti ugovor ili mapiranje.

Završetak putanje sada možemo izraziti dokazima: Network sadrži zahtjev i odgovor, backend zapis pokazuje obrađenu rutu, a komponenta sadrži podatke koji odgovaraju prikazu. Jedan dokaz ne zamjenjuje druge. Vratimo se početnom spinneru: ista vanjska slika mogla je nastati prije slanja ili nakon uspješnog odgovora, pa nam upravo ovaj slijed govori gdje tražiti razliku.

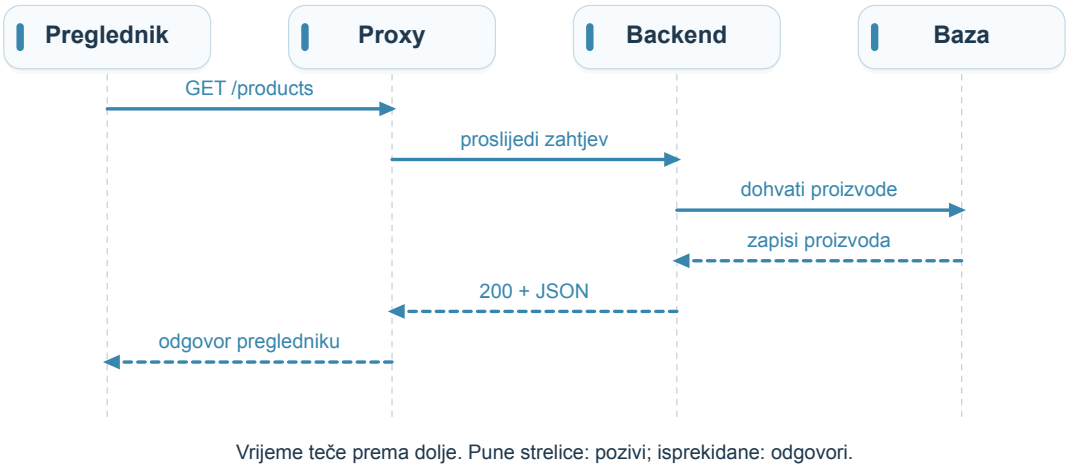


Figure 11: Potpuni put zahtjeva i odgovora kroz četiri komponente, uključujući povratak u preglednik.

Preglednik prima status i tijelo.

Kod provjerava odgovor, čita JSON i ažurira stanje.

Tek tada komponenta može prikazati proizvode.

> **Za nastavnika:** Gdje tražimo problem ako Response ima proizvode, a prikaz ih nema?
>
> **Očekivano obrazloženje:** U parsiranju, ugovoru podataka, stanju i prikazu; ne zaključujemo odmah da je baza kriva.

34 Gdje je stanje?

> **Vrijeme za nastavu:** 86–89 min (3 min).

Naša aplikacija istodobno sadrži više oblika stanja. U pregledniku su prikazani proizvodi i oznaka čekanja. U backendu može postojati privremeni rezultat upita. U bazi su trajno pohranjeni zapisi. Isti proizvod može biti predstavljen na sva tri mjesta, ali kopije ne moraju imati istu aktualnost.

Ako se backend ponovno pokrene, varijabla u njegovoj memoriji ne postaje automatski trajna. Ako korisnik zatvori karticu, to ne briše proizvode iz baze. Ako baza izgubi podatke, preostali prikaz u pregledniku nije dokaz uspješne trajne pohrane. Za ovu raspravu pretpostavljamo uobičajenu trajnu konfiguraciju baze, a ne tvrdimo da svaki način pokretanja baze automatski pruža istu trajnost. Ograničenja i oporavak razradit ćemo kasnije.

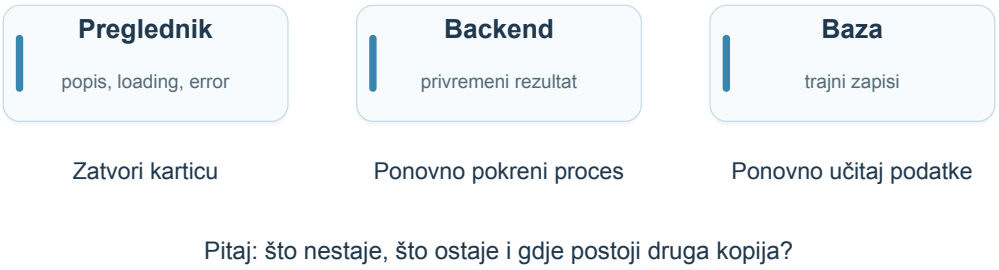


Figure 12: Tri mjesta stanja: preglednik, memorija backenda i trajni zapisi baze.

Zatvaranje kartice, restart procesa i gubitak podataka nisu isto.

Za svaku vrijednost pitajte: tko je čuva, koliko dugo i gdje postoji druga kopija?

> **Za nastavnika:** Koja bi vrijednost nestala pri restartu samo backend procesa?
>
> **Očekivano obrazloženje:** Njegova privremena memorija; ne nužno trajni podaci baze niti stanje otvorenog preglednika.

35 Što znači HTTP stateless?

> **Vrijeme za nastavu:** 89–92 min (3 min).

Izraz stateless opisuje HTTP semantiku, a ne zabranu pohrane u aplikaciji. Poslužitelj ne mora iz prethodnog HTTP razgovora rekonstruirati koju operaciju predstavlja nova poruka. Aplikacija može u poruci dobiti identifikator sesije i preko njega pronaći spremljeno stanje; kako taj identifikator putuje, pokazuje sljedeći odjeljak o cookieju.

Uzmimo dvije kartice istog korisnika. Svaka šalje zahtjev za proizvode. Ako oba zahtjeva nose potreban kontekst, backend ih može tumačiti zasebno, dok baza i dalje čuva proizvode i korisnike. Tvrdnja „HTTP je stateless, pa server ništa ne pamti” brka protokol s dizajnom aplikacije. Druga česta zabuna je „stateless znači da se veza nakon svake poruke zatvara”; to je zasebno pitanje upravljanja vezom.

Definicija 1.1

HTTP je **stateless**: značenje zahtjeva ne oslanja se na obvezan prethodni dijalog koji HTTP pamti za tog klijenta.

Aplikacija i dalje može imati **korisnike, košarice i sesije**.

Izvori: RFC 9110: HTTP Semantics.

Za nastavnika: Je li spremljena košarica dokaz da HTTP nije stateless?

Očekivano obrazloženje: Ne. To je aplikacijsko stanje, ne obvezni dijalog HTTP protokola.

36 Cookie prenosi kontekst

Vrijeme za nastavu: 92–95 min (3 min).

Dosad smo dohvaćali javne proizvode. Zamislimo sada prijavljenog korisnika: kako aplikacija povezuje više njegovih zahtjeva? Korisnička sesija je spremljeni kontekst, primjerice kojoj prijavi pripadaju zahtjevi. Jedan način povezivanja je identifikator sesije spremljen u cookieju. Cookie je mala vrijednost koju preglednik pamti i šalje uz odgovarajuće zahtjeve; ne mora služiti prijavi.

Poslužitelj preko Set-Cookie može zatražiti da preglednik zapamti vrijednost, a preglednik je pod odgovarajućim uvjetima šalje u Cookie zaglavlju. abc123 je izmišljena nastavna oznaka; stvarni identifikator mora biti prikladno generiran. Primjer izostavlja atribute radi čitanja puta vrijednosti, a nije konfiguracija prijave za produkciju. Primjer koristi neprozirni identifikator; ne tvrdi da cookie mora sadržavati korisničke podatke ili JWT. Aplikacija može preko identifikatora dohvatiti sesiju iz odabrane pohrane.

Secure ograničava uobičajeno slanje na sigurnu vezu, HttpOnly ograničava pristup iz JavaScripta, a SameSite utječe na slanje u međumjesnim kontekstima. To nisu zamjene za provjeru dopuštenja. Danas je dovoljno prepoznati put vrijednosti. Cjelovitu prijavu, odjavu, pohranu sesije i napade ostavljamo za predavanja o identitetu i sigurnosti.

```
Set-Cookie: sid=abc123
```

Kasniji prikladni zahtjev može sadržavati:

```
Cookie: sid=abc123
```



Figure 13: Cookie nosi identifikator sesije u svakom zahtjevu; backend preko njega pronalazi spremljeno stanje. Izmišljen primjer.

Cookie je mehanizam prijenosa; pravila sesije određuje aplikacija.

Izvori: MDN: Using HTTP cookies.

> **Za nastavnika:** Dokazuje li samo prisutnost sid cookieja da je korisnik ovlašten za tuđi zapis?
>
> **Očekivano obrazloženje:** Ne. Server provjerava sesiju i dopuštenja za konkretnu operaciju.

37 Jedna TCP veza, više HTTP zahtjeva

> **Vrijeme za nastavu:** 95–97 min (2 min).

Kad smo `nc` naredbom poslali zahtjev, otvorili smo TCP vezu. Nakon odgovora ta veza nije nužno zatvorena. U HTTP/1.1 veza prema zadanom ostaje otvorena (*persistent connection*, često zvana *keep-alive*), pa isti klijent kroz nju može poslati sljedeći zahtjev. Time se izbjegava trošak ponovne uspostave veze, ali se ne mijenja značenje zahtjeva: svaki je i dalje zasebna, potpuna poruka.

To možemo provjeriti na našem poslužitelju. U istoj `nc` vezi pošaljemo `GET /products`, a odmah nakon odgovora i `GET /products?case=empty`. Oba odgovora stižu kroz istu vezu. U zaglavljima odgovora vidimo:

```
Connection: keep-alive
Keep-Alive: timeout=5
```

Zaglavlje `Connection: keep-alive` potječe iz HTTP/1.0, gdje je trajnu vezu trebalo izričito zatražiti; Node ga šalje i u HTTP/1.1. `timeout=5` znači da poslužitelj neaktivnu vezu zatvara nakon pet sekundi. To je zadana vrijednost Nodea (`server.keepAliveTimeout`), a ne pravilo protokola. Preglednik pak sam odlučuje koliko će veza otvoriti i koliko će ih dugo držati.

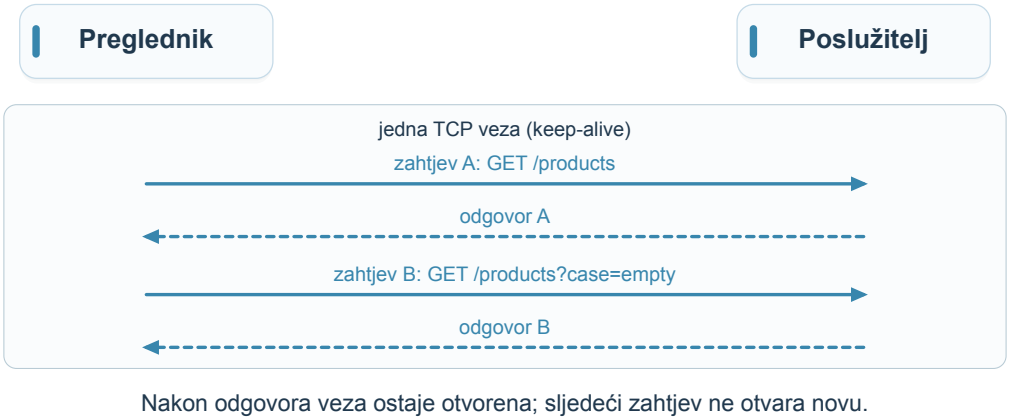


Figure 14: Dva zahtjeva i dva odgovora unutar iste otvorene TCP veze.

Slika prikazuje uzastopne poruke HTTP/1.1. HTTP/2 kroz jednu vezu može istodobno prenositi više zahtjeva, a HTTP/3 ne koristi TCP; to ne mijenjamo u današnjem modelu.

Iz toga slijedi važno razgraničenje: **veza nije prijava korisnika.** Veze se otvaraju i zatvaraju iz mrežnih razloga, a jedan korisnik u pregledniku obično koristi više veza. Ako proxy stoji ispred backenda, backend vidi vezu s proxyjem, a ne s korisnikom. Zato aplikacija korisnika prepoznaje po podacima koje šalje u samom zahtjevu (primjerice kolačiću), o čemu će biti riječi u kasnijim predavanjima.

Izvori: MDN: Connection management in HTTP/1.x; MDN: Keep-Alive; Node.js: server.keepAliveTimeout.

> **Za nastavnika:** Znači li nova TCP veza novu prijavu korisnika?
>
> **Očekivano obrazloženje:** Ne. Identitet korisnika aplikacija prepoznaje iz podataka u zahtjevu, ne iz veze.

38 CORS: odgovor i pristup iz JavaScripta

> **Vrijeme za nastavu:** 97–98 min (1 min).

Vratimo se dohvatu proizvoda: što ako frontend i backend posluživaju različite adrese? Podrijetlo (origin) određuju način pristupa, ime domaćina i port. CORS (Cross-Origin Resource Sharing) je mehanizam kojim poslužitelj kroz zaglavlja dopušta pregledniku određeni pristup preko granice podrijetla.

Kod lokalnog frontenda i backenda različiti portovi znače različita podrijetla. `localhost` je ime za ovo računalo, adresu `127.0.0.1`; 5173 je zadani port razvojnog poslužitelja Vite, koji poslužuje frontend, a 3000 port našeg backenda. Preglednik zato može zahtijevati odgovarajuća CORS zaglavlja kako bi JavaScript smio čitati odgovor. Za neke zahtjeve prije stvarne operacije šalje provjerni preflight. Ne tvrdimo da se svaki CORS-neuspjeh događa prije slanja: ovisi o vrsti zahtjeva i provjeri.

U dijagnozi stoga razdvajamo backend koji je obradio zahtjev od koda koji ne može pročitati odgovor. Uspjeh istog poziva iz alata izvan preglednika ne znači da je preglednička politika nevažna. S druge strane, dopušten CORS ne daje korisniku pravo na tuđe podatke. Za naš jednostavni demo frontend i API dolaze iz istog lokalnog podrijetla, kako bi osnovni put ostao pregledan.

Podrijetlo čine **shema, naziv domaćina i port**.

`http://localhost:5173` `http://localhost:3000`

- Preglednik ograničava čitanje odgovora drugog podrijetla.
- CORS određuje dopušten pristup.
- CORS **ne provjerava tko je korisnik ni smije li čitati podatke**.

Izvori: MDN: Cross-Origin Resource Sharing.

> **Za nastavnika:** Može li izravan klijent poslati zahtjev iako mu CORS ne dopušta browser pristup?
>
> **Očekivano obrazloženje:** Da. Backend treba vlastite provjere; CORS nije zaštita od svih klijenata.

39 CORS na primjeru: zahtjev stiže, čitanje ne

> **Vrijeme za nastavu:** 98–99 min (1 min).

Pogledajmo CORS na vlastitom poslužitelju. Pokrenemo dva procesa istog programa: jedan na portu 3000, drugi na portu 3001. Otvorimo stranicu `http://127.0.0.1:3001` i iz njezina JavaScripta pozovemo `fetch('http://127.0.0.1:3000/products')`. Podrijetla se razlikuju u portu, kao frontend na 5173 i backend na 3000.

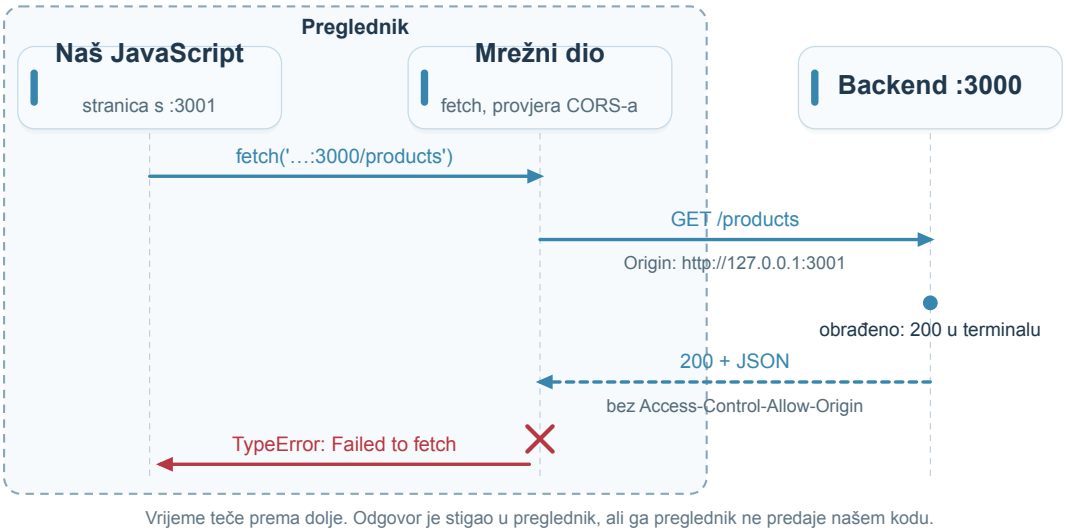


Figure 15: Slijed CORS zahtjeva: zahtjev stiže do backenda, a preglednik odgovor ne predaje JavaScriptu.

Slijed ima pet koraka:

1. JavaScript poziva `fetch`.
2. Preglednik šalje `GET /products` i sam dodaje zaglavlje `Origin: http://127.0.0.1:3001`: podrijetlo stranice s koje kod šalje zahtjev.
3. Backend obradi zahtjev i u terminalu zapiše status 200.
4. Odgovor stiže do preglednika, ali nema zaglavlje `Access-Control-Allow-Origin`.
5. Preglednik odgovor ne predaje JavaScriptu: `fetch` završava s `TypeError: Failed to fetch`, a konzola javlja da je pristup blokiran CORS politikom.

Iz frontenda to izgleda kao mrežna greška, ali backend je posao obavio. Zato dijagnozu ne završavamo u konzoli preglednika nego provjerimo i zapis backenda. Čitanje dopušta backend, zaglavljem `Access-Control-Allow-Origin: http://127.0.0.1:3001` u odgovoru; frontend to ne može sam riješiti.

Ovo je jednostavan GET zahtjev pa ga preglednik šalje odmah. Za druge zahtjeve, primjerice s JSON tijelom ili vlastitim zaglavljima, preglednik najprije šalje provjerni `OPTIONS` zahtjev (*preflight*) i bez dopuštenja stvarni zahtjev ne šalje. To ćemo razraditi kad backend počne primati podatke.

Ponašanje je provjereno u headless Chromeu s nastavnim poslužiteljem iz `code/01-introduction`.

Izvori: MDN: Cross-Origin Resource Sharing.

> **Za nastavnika:** Je li backend vratio grešku?
>
> **Očekivano obrazloženje:** Ne. Vratio je 200; čitanje je blokirao preglednik jer odgovor ne dopušta podrijetlo :3001.

40 Kolačići na drugo podrijetlo

Vrijeme za nastavu: 99–101 min (2 min).

Zadana vrijednost opcije `credentials` u pozivu `fetch` je `"same-origin"`: preglednik šalje kolačiće i sprema `Set-Cookie` samo kad zahtjev ide na isto podrijetlo. Ako frontend na `http://localhost:5173` poziva backend na `http://localhost:3000`, kolačić sesije se ne šalje i backend ne zna tko je korisnik. Da bi ga slao, naš kod mora to izričito zatražiti:

```
fetch("http://localhost:3000/cart", {
  credentials: "include",
});
```

`credentials` nije HTTP zaglavlje i ne putuje mrežom. To je uputa pregledniku smije li zahtjevu dodati zaglavlje `Cookie` i spremiti kolačić iz odgovora. Vrijednost `"omit"` nikad ne šalje kolačiće. Biblioteka axios za isto koristi opciju `withCredentials: true`; zadana vrijednost `false` ponaša se kao `"same-origin"`.

Da bi naš kod smio pročitati takav odgovor, backend mora poslati dva zaglavlja:

```
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Credentials: true
```

Uz kolačiće `*` nije dopušten: podrijetlo mora biti navedeno točno. Ako bilo što od toga nedostaje, `fetch` završava s `TypeError`, kao kod običnog CORS-a.

Važno je što se dogodilo prije te provjere. Kolačić je poslan već zbog `credentials: "include"`, a backend je zahtjev obradio. CORS zaglavlja odlučuju samo smije li naš kod pročitati odgovor. Zato backend ne smije pretpostaviti da zahtjev s ispravnim kolačićem dolazi s njegova frontenda; o tome više u predavanju o sigurnosti.

Izvori: MDN: RequestInit credentials; MDN: Access-Control-Allow-Credentials.

> **Za nastavnika:** Frontend šalje `credentials: "include"`, a backend vraća `Access-Control-Allow-Origin: *` i `Access-Control-Allow-Credentials: true`. Što dobiva naš kod i je li backend obradio zahtjev?
>
> **Očekivano obrazloženje:** Naš kod dobiva `TypeError` jer uz kolačiće `*` ne vrijedi. Backend je ipak dobio kolačić i obradio zahtjev.

41 Simulator: kolačić i CORS

Vrijeme za nastavu: 101–103 min (2 min).

Kolačić i CORS najlakše je razlikovati kad ih promatramo na istim zahtjevima. U HTML prezentaciji ovog predavanja nalazi se simulator: odaberemo tko šalje zahtjev prema `http://localhost:3000` (stranica s istog podrijetla, stranica s `http://localhost:5173` ili `curl`), što backend šalje u `Access-Control-Allow-Origin` te jedan od zahtjeva `GET /products`, `POST /login` i `GET /cart`. Simulator prikazuje četiri koraka: zahtjev koji preglednik šalje, obradu na backendu, odgovor i ono što dobiva naš JavaScript.

Simulator je pojednostavljen model. Pokriva samo jednostavne zahtjeve, bez preflighta. Opcija `credentials` u pozivu `fetch` određuje šalje li preglednik kolačiće i prihvaća li `Set-Cookie`: `'same-origin'` (zadano) samo za isto podrijetlo, `'include'` i za drugo podrijetlo, `'omit'` nikad.

Šest pokusa vrijedi predvidjeti prije klika:

1. **Drugo podrijetlo, bez dopuštenja.** Backend vraća 200, a naš kod dobiva `TypeError: Failed to fetch`. Zahtjev je stigao; preglednik nije predao odgovor.
2. **Pogrešno dopušteno podrijetlo.** `Access-Control-Allow-Origin: http://127.0.0.1:5173` ne dopušta stranicu s `http://localhost:5173`: ime domaćina je drugačije, iako oba imena vode na isto računalo. Podrijetlo se mora točno podudarati.
3. **Prijava na istom podrijetlu.** Odgovor na `POST /login` sadrži `Set-Cookie: sid=abc123`; preglednik ga spremi i sam doda `Cookie: sid=abc123` u sljedeći `GET /cart`. Backend preko te vrijednosti pronalazi sesiju.
4. **Dopušten CORS, ali bez kolačića.** Sa stranice na `http://localhost:5173` i uz ispravno dopuštenje naš kod čita odgovor, ali to je `401`: `fetch` na drugo podrijetlo prema zadanom ne šalje kolačiće, pa backend ne zna čija je košarica. CORS ne prijavljuje korisnika.
5. **Kolačići na drugo podrijetlo.** Uz `credentials: 'include'` preglednik šalje `Cookie: sid=abc123` i backend vraća košaricu. Naš kod je čita tek uz točno podrijetlo i `Access-Control-Allow-Credentials: true`, kako je opisano u prethodnom odjeljku.
6. **Izravni klijent.** `curl` čita odgovor bez obzira na CORS zaglavlja, jer CORS provodi preglednik. Backend zato ne smije računati na CORS kao zaštitu podataka.

Kolačić je vezan uz ime domaćina, a ne uz port. Atribut `SameSite` i zaštitu od lažiranih zahtjeva (CSRF) obrađujemo u predavanju o sigurnosti.

Izvori: MDN: Cross-Origin Resource Sharing; MDN: Using HTTP cookies; MDN: RequestInit credentials.

> **Za nastavnika:** Prije svakog klika pitajte što će se dogoditi u koracima 1 i 4. Kod pokusa 6 pitajte: može li izravni klijent pročitati odgovor iako CORS ne dopušta pristup iz preglednika?
>
> **Očekivano obrazloženje:** Da. CORS provodi preglednik; backend treba vlastite provjere.

42 Gdje odlazi vrijeme?

Vrijeme za nastavu: 103–105 min (2 min).

Vrijeme korisničkog doživljaja uključuje pripremu veze kada je potrebna, prijenos, čekanje obrade i baze, čitanje tijela te rad prikaza. TTFB (Time to First Byte) je vrijeme do prvog primljenog bajta odgovora; nije čisto mjerenje CPU vremena backenda. Mreža, posrednici i čekanje utječu na opažanje.

Na slici duljine segmenata služe objašnjenju, a nisu rezultat mjerenja. Za usporedbu postavimo vlastiti eksperiment: dva odgovora s jednakim brojem proizvoda, ali jedan s namjerno dodanim čekanjem od dvije sekunde. Ako server izmjeri malu obradu, a klijent dugo čeka, treba istražiti granice izvan tog mjerenja. Ako velik dio vremena troši ažuriranje tisuća elemenata, promjena načina dohvata iz baze ne rješava izravno taj problem.

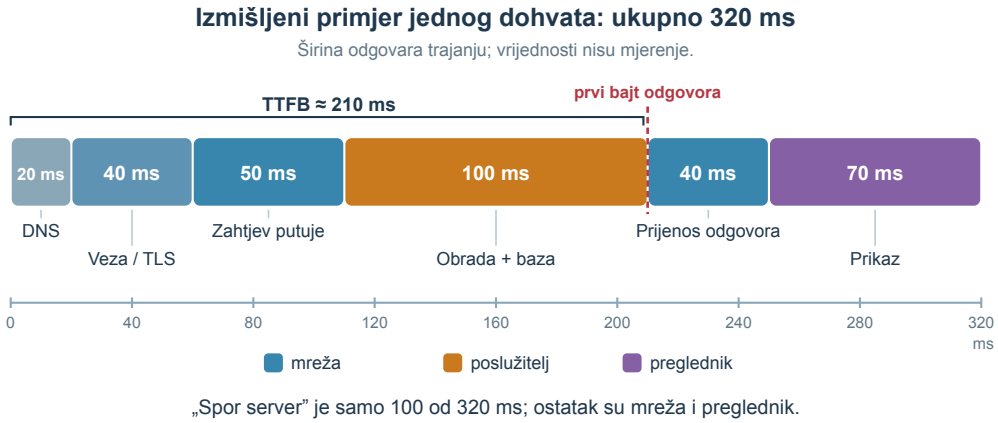


Figure 16: Izmišljeni vodopad jednog dohvata, ukupno 320 ms: DNS 20, veza/TLS 40 i zahtjev putuje 50 (mreža), obrada i baza 100 (poslužitelj), prijenos odgovora 40 (mreža), prikaz 70 (preglednik); TTFB oko 210 ms.

TTFB (*Time to First Byte*) je vrijeme do prvog bajta odgovora.

Mjerimo određenu granicu; ne nazivamo sve „sporim serverom".

Izvori: MDN: Time to First Byte; MDN: Connection management in HTTP/1.x.

Za nastavnika: Je li vrijeme do prvog bajta jednako vremenu dohvata iz baze?

Očekivano obrazloženje: Ne. Obuhvaća širi put do prvog bajta.

43 Bez odgovora: što se doista dogodilo?

Vrijeme za nastavu: 105–108 min (3 min).

U ovoj varijanti backend je završio dohvat, ali odgovor nije u cijelosti stigao pregledniku. Korisnik vidi neuspjeh, premda je poslužitelj obavio čitanje. Moguća je i drukčija varijanta: zahtjev nikada nije stigao. Iz istog vanjskog opažanja ne smijemo proizvoljno odabrati jedan uzrok.

GET u našem primjeru samo čita proizvode, pa ponovno čitanje ne treba stvoriti narudžbu. To svojstvo ovisi o tome da implementacija poštuje semantiku dohvata. Kod budućeg POST /orders posljedice ponavljanja bit će mnogo važnije. Danas ne uvodimo cijeli protokol ponavljanja; učimo razlikovati opažanje klijenta od dovršenog rada drugog sudionika i potrebu za dodatnim dokazima.

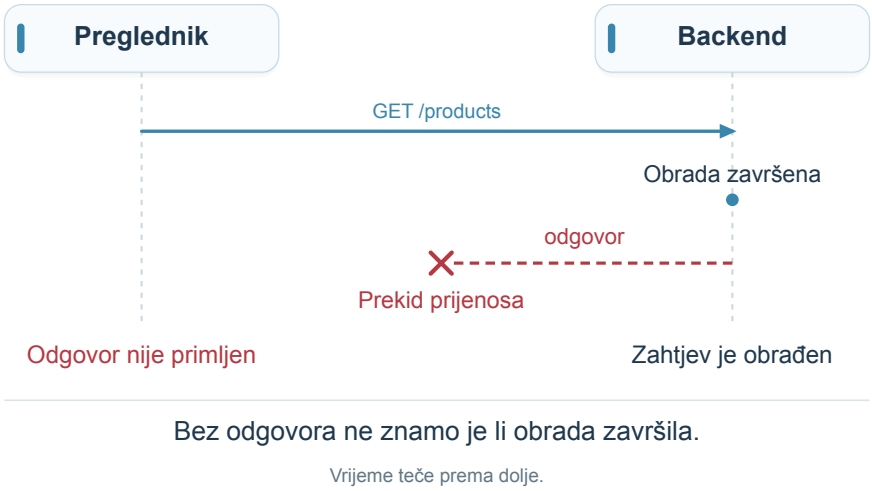


Figure 17: Backend je obradio zahtjev, ali odgovor ne stiže pregledniku; vrijeme teče prema dolje.

Zahtjev može biti obrađen, a odgovor nečitljiv ili izgubljen.

Istek čekanja govori da nismo dobili rezultat na vrijeme.

Ne dokazuje da backend nije radio.

> **Za nastavnika:** Koji dokaz bi razlikovao izgubljen zahtjev od izgubljenog odgovora?
>
> **Očekivano obrazloženje:** Povezani zapisi primitka i završetka obrade na serveru, uz oprez: odsutnost loga nije uvijek dokaz odsutnosti događaja.

44 Vježba: locirajte sljedeću provjeru

> **Vrijeme za nastavu:** 108–111 min (3 min).

Parovi dobiju dvije minute da za svaki redak predlože sljedeću provjeru. Ne tražimo konačnu dijagnozu na temelju nedovoljnih podataka. Za prvi slučaj provjeravaju poziva li se funkcija, je li došlo do ranije JavaScript greške i filtrira li Network prikaz. Za prazan niz provjeravaju ugovor i stvarni podatkovni rezultat. Za primljene proizvode pregledavaju mapiranje, stanje i prikaz.

Za 504 prvo treba utvrditi koji je posrednik odgovorio i povezati njegove zapise s backendom. Kod CORS-a treba pregledati podrijetla, preflight ako postoji i zaglavlja, bez zaključka da poziv nije mogao stići. Nakon rada po jedan par brani rješenje, a drugi navodi alternativno objašnjenje. Vrednuje se kvaliteta dokaza, ne brzina imenovanja komponente.

Opažanje	**Gdje biste dalje gledali?**
Nema `/products` u Networku	?
`200`, tijelo `[]`	?
`200`, proizvodi postoje, prikaz prazan	?
Proxy vraća `504`	?
Backend ima rezultat, preglednik javlja CORS	?

U paru: **hipoteza + dokaz koji je može opovrgnuti**.

> **Za nastavnika:** Za svaki redak odaberite jednu provjeru i jedno alternativno objašnjenje.
>
> **Očekivano obrazloženje:** Prihvatiti različite provjere ako doista razlikuju hipoteze; tražiti opažanje koje bi promijenilo zaključak.

45 Provjera: tri česte zablude

> **Vrijeme za nastavu:** 111–113 min (2 min).

Tražimo protuprimjere, a ne samo riječ „netočno”. Prva tvrdnja pada kada aplikacija koristi identifikator sesije i spremljenu košaricu: HTTP i dalje prenosi samostalne poruke, dok aplikacija čuva podatke. Druga pada kod šifriranog zahtjeva za tuđom narudžbom koji treba odbiti autorizacijom. Treća pada kada tijelo sadrži valjane proizvode, ali klijent čita pogrešno polje.

Dodatno pitajte može li ista TCP veza prenijeti više zahtjeva. Student koji jasno razdvoji transport, protokol, aplikacijska pravila i prikaz ima osnovu za iduća predavanja. Ako odgovor ostane samo na terminima, zatražite da pokaže okvir ili strelicu na karti na koju se tvrdnja odnosi.

1. „HTTP je stateless pa aplikacija ne može pamtiti košaricu.”
2. „HTTPS znači da je korisnik ovlašten.”

3. „Server je vratio 200 pa se stranica ispravno prikazala.”

Za svaku tvrdnju ponudite **konkretan protuprimjer**.

> **Za nastavnika:** Navedite protuprimjer i komponentu koja određuje ishod.
>
> **Očekivano obrazloženje:** Sesija/baza; backend ovlasti; klijentski prikaz. Ne pripisivati jamstvo pogrešnom sloju.

46 Sažetak jednog zahtjeva

> **Vrijeme za nastavu:** 113–115 min (2 min).

Vratimo se početnom pitanju. Sada možemo objasniti zašto otvorena stranica ne dokazuje dostupnost svih komponenti i zašto prazan prikaz ne dokazuje praznu bazu. Umjesto jedne opće tvrdnje slažemo lanac opažanja: slanje, primitak, obrada, odgovor i prikaz. Na svakoj granici postoji konkretna poruka ili lokalna promjena stanja.

Sažetak nije popis novih pojmova za pamćenje. On je postupak za nepoznatu aplikaciju: nacrtati sudionike, imenovati podatke, zapisati pretpostavke i predložiti provjeru. Taj postupak ostaje koristan i ako Node.js zamijenimo drugim backendom ili promijenimo bazu. Proizvod demonstrira načelo, ali ne zamjenjuje objašnjenje načela.

Pronađi odredište → pošalji → obradi → vrati → prikaži.

- Poruka ima namjeru, metapodatke i sadržaj.
- Stanje postoji na više mjesta.
- Veza, HTTP i sesija različiti su slojevi.
- Kašnjenje i kvar vežemo uz konkretnu granicu.

Početni spinner nije bio dovoljna dijagnoza.

> **Za nastavnika:** Objasnite put GET /products bez naziva frameworka.
>
> **Očekivano obrazloženje:** Prihvatiti precizan opis odgovornosti, poruka i granica.

47 Osmislite sljedeći korak

> **Vrijeme za nastavu:** 115–118 min (3 min).

Studenti raspoređuju poslove narudžbe između preglednika, backenda i baze. Isti posao smije biti na više mjesta s različitom svrhom: provjera unosa u pregledniku pomaže korisniku, a ona u backendu štiti podatke. Tražite razlog za svako razdvajanje: što postaje lakše provjeriti ili promijeniti, a koja složenost nastaje.

Nakon početnog prijedloga dodajte sporu bazu i ponovni klik. Ne očekujemo cjelovito rješenje transakcija i idempotentnosti u prvom tjednu. Dovoljno je prepoznati da onemogućen gumb nije jedina zaštita i da server mora odrediti značenje ponavljanja. Završite usporedbom dvaju obranjivih prijedloga. Drugo predavanje preuzima upravo ovaj rastući handler i razrađuje arhitekturu backenda.

Dodajemo naručivanje: `POST /orders` šalje podatke za novu narudžbu.

Posao	Preglednik	Backend	Baza
provjeriti unos (e-pošta, količina)	?	?	?
odrediti cijenu	?	?	?
provjeriti ima li dovoljno na zalihi	?	?	?
spremiti narudžbu	?	?	?

Tko radi koji posao? Smije li isti posao biti na više mjesta?

Dodatni uvjet: baza kasni, a korisnik ponovno klikne **Naruči**.

> **Za nastavnika:** Parovi popune tablicu; više odgovora je obranjivo, tražite razlog za svaki.
>
> **Očekivano obrazloženje:** Unos – preglednik pomaže korisniku, backend mora provjeriti jer klijent šalje što hoće. Cijena – backend iz pohranjenih podataka, nikad iz zahtjeva. Zaliha – backend provjerava, a baza može dodatno čuvati ograničenje (zaliha ne smije biti negativna). Spremanje – backend traži, baza trajno pohranjuje. Onemogućen gumb pomaže, ali nije zaštita; značenje ponovljenog zahtjeva određuje backend (5. i 9. predavanje).

48 Pregled: 10 zaglavlja zahtjeva

> **Vrijeme za nastavu:** 118–118.5 min (0.5 min).

Smjer: preglednik → poslužitelj. Ovo je pregled za prepoznavanje u Networku, ne popis za učenje napamet. Ne tražite razumijevanje svih mehanizama unaprijed; kolačiće, CORS i predmemoriju razrađujemo kasnije.

Zaglavlje	Značenje
`Host`	Odredište zahtjeva u HTTP/1.1.
`Accept`	Koje formate odgovora klijent prihvaća.
`Accept-Language`	Željeni jezici odgovora.
`Accept-Encoding`	Prihvatljivi načini sažimanja odgovora.

Zaglavlje	Značenje
`Content-Type`	Format tijela koje klijent šalje.
`Authorization`	Podaci za provjeru identiteta/pristupa.
`Cookie`	Ranije spremljene vrijednosti za ovo odredište.
`Origin`	Podrijetlo zahtjeva: shema, host i port.
`Referer`	Adresa stranice s koje zahtjev potječe, ako se šalje.
`User-Agent`	Podaci o klijentskom programu.

Zaglavlja nisu obvezan komplet. Njihova prisutnost ovisi o poruci, pregledniku i poslužitelju. Content-Type opisuje tijelo u oba smjera. Dio zahtjevnih zaglavlja postavlja preglednik; nije ih sve moguće proizvoljno postaviti u fetch pozivu. Host je ovdje prikazan za HTTP/1.1; novije verzije koriste :authority. Content-Length se odnosi na sadržaj poruke, ne veličinu JavaScript objekta.

Izvor: MDN: zaglavlja.

49 Pregled: 10 zaglavlja odgovora

Vrijeme za nastavu: 118.5–119 min (0.5 min).

Smjer: poslužitelj → preglednik. Ovo je pregled za prepoznavanje u Networku, ne popis za učenje napamet. Ne tražite razumijevanje svih mehanizama unaprijed; kolačiće, CORS i predmemoriju razrađujemo kasnije.

Zaglavlje	Značenje
`Content-Type`	Format tijela odgovora.
`Content-Length`	Veličina sadržaja u bajtovima, kad je navedena.
`Content-Encoding`	Kako je sadržaj sažet, npr. gzip.
`Cache-Control`	Pravila spremanja i ponovne uporabe odgovora.
`ETag`	Oznaka verzije sadržaja za kasniju provjeru.
`Last-Modified`	Vrijeme posljednje promjene sadržaja.
`Location`	Adresa za preusmjeravanje ili novostvoreni resurs.
`Set-Cookie`	Uputa pregledniku za spremanje kolačića.
`WWW-Authenticate`	Način provjere identiteta koji poslužitelj traži.

Zaglavlje	Značenje
`Access-Control-Allow-Origin`	Koje podrijetlo smije čitati odgovor u pregledniku.

Zaglavlja nisu obvezan komplet. Njihova prisutnost ovisi o poruci, pregledniku i poslužitelju. Content-Type opisuje tijelo u oba smjera. Dio zahtjevnih zaglavlja postavlja preglednik; nije ih sve moguće proizvoljno postaviti u fetch pozivu. Host je ovdje prikazan za HTTP/1.1; novije verzije koriste :authority. Content-Length se odnosi na sadržaj poruke, ne veličinu JavaScript objekta.

Izvor: MDN: zaglavlja.